

VIVRITI CAPITAL PRIVATE LIMITED

*Submitted in partial fulfillment of the
requirement for the award of the
Degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING**

by

PANDURI VISHNU VARDHAN (21BCE9349)

Under the Guidance of

THANGA AMRITHAKUMAR A



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237**

MAY 2025

DECLARATION

I hereby declare that the thesis entitled “**VIVRITI CAPITAL PRIVATE LIMITED**” submitted by me, for the award of the degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING AT VIT-AP UNIVERSITY** is a record of bonafide work carried out by me under the supervision of **THANGA AMRITHAKUMAR A.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravathi

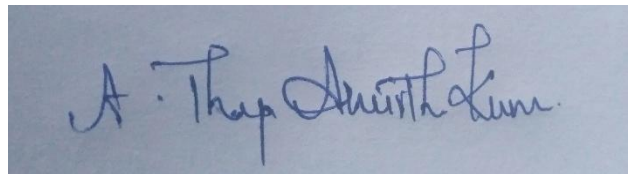
Date:22/05/2025

A handwritten signature in black ink, reading "P. Vishnuvardhan" with a stylized flourish at the end.

Signature of the Candidate

CERTIFICATE

This is to certify that the Internship Project titled “**VIVRITI CAPITAL PRIVATE LIMITED**” that is being submitted by PANDURI VISHNU VARDHAN (21BCE9349) is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other institute or university for award of any degree or diploma and the same is certified.

A rectangular box containing a handwritten signature in blue ink. The signature appears to read "A. Thanga Amritha Kumar".

Thanga AmrithaKumar A
Guide

The thesis is satisfactory

Approved by

Dr.G.Muneeswari

**HoD, Department of Computer Science and Engineering School of
Computer Science and Engineering**

ABSTRACT

During my internship at Vivriti, I cultivated a comprehensive understanding of Spring Boot and its ecosystem, mastering key concepts and tools vital for developing robust Java applications. I delved into Spring Core fundamentals, such as Inversion of Control (IoC) and Dependency Injection, utilizing BeanFactory, ApplicationContext, and annotations like @Autowired to manage beans efficiently. I became adept at configuring Spring Beans through constructor and setter injection, comprehending bean lifecycles, and applying scopes like Singleton and Prototype. My learning encompassed Spring Boot's architecture, leveraging features like auto-configuration, actuators, and starters (e.g., Starter Web, Starter Test) to enhance development efficiency. I built RESTful APIs, validated endpoints with Postman, and implemented CRUD operations using Spring Data JPA and MongoRepository, integrating with databases like MySQL, MariaDB, and MongoDB. I also looked into Spring Security to handle authentication, used Jasypt for encrypting passwords securely, and employed ModelMapper to make DTO mapping more efficient. My experience with Spring Batch helped me process large amounts of data, while Spring Kafka supported message-driven microservices, enabling us to manage JSON and string messages through Apache Kafka.

I set up scheduling, logging, and aspect-oriented programming (AOP) to handle common concerns like cross-cutting issues, which made the application more modular. Also, I worked with Docker to containerize applications, learning how to write Dockerfiles and set up port mappings for scalable deployment. A big part of my work was mastering Drools, the business rules management system. I gained skills in configuring KieServices, KieContainer, and KieSession to run rules dynamically. I also learned how to write rules using Java dialect, making sure they integrated smoothly with Spring Boot apps. During this process, I tackled challenges like dependency conflicts and classloader problems by aligning drools-core, drools-mvel, and kie-api dependencies. I improved rule processing performance by moving from outdated APIs like StatefulKnowledgeSession to the newer KieSession. My work included connecting rules to MongoDB for flexible storage and using KnowledgeBuilder for faster rule compilation. I also focused on optimizing performance by using techniques like EntityGraph to reduce database queries and speed up responses. This experience also sharpened my debugging skills, especially in fixing issues like NullPointerExceptions and ClassCastExceptions by improving logging and dependency management. Overall, this internship gave me a strong foundation in Spring Boot, Drools, and related technologies, preparing me for complex software projects that are scalable, secure, and efficient.

What's more, I expanded my skills with advanced tools and frameworks. I applied Aspect-Oriented Programming (AOP) to handle cross-cutting concerns such as logging and transaction management, using annotations like @Before, @After, and @Around to improve modularity. Spring Boot Actuator was useful for monitoring application health and metrics in real time. I used JSONPath to query JSON data more efficiently, which helped optimize API responses. My work with TimeSeriesDB allowed me to manage time-series data effectively, while FastExcel and Apache POI supported high-performance Excel report generation and data processing. Spring Batch strengthened my ability to handle batch processing pipelines, ensuring reliability. MongoDB integration enhanced my proficiency in NoSQL database operations, and Spring DevTools accelerated development through features like automatic restarts

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to **THANGA AMRITAKUMAR A, Tech Team Lead - Vivriti Capital**, for his/her constant guidance, continual encouragement, understanding; more than all, she taught me patience in my endeavor. My association with him/her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of Machine Learning.

I would like to express my gratitude to **Dr. G. Viswanathan, Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, and Dr. G. V. Selvam, Vice Presidents (VPs), Dr. S. V. Kota Reddy, Vice-Chancellor (VC), and Dr. S. Sudhakar Ilango, Dean, School of Computer Science and Engineering (SCOPE)**, for providing me with an environment to work in and for their inspiration during the tenure of the course.

In a jubilant mood, I express ingeniously my whole-hearted thanks to **MAHENDRA RAHUL CR, HEAD ENGINEERING**, all technical staff, and members working as limbs of our company for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully.

I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Amaravathi

Date:22/05/2025

Panduri Vishnu Vardhan

[Name of the Student]

TABLE OF CONTENTS

Sl.No.	Chapter	Title	Page Number
1.		Abstract	3
2.		Acknowledgement	4
3.		List of Figures and Table	6
4.		List of Arconyms	7
5.	1	Introduction	8
	1.1	Objectives	9-11
	1.2	Background and Literature Survey	12-20
	1.3	Proposed Enhancements	21-23
	1.4	Organization of the Report	24-27
	1.5	Scope of the project	27-31
	1.6	Challenges present in the market	27-31
6.	2	Chapter Title (Work)	
	2.1	Proposed System	32
	2.2	System architecture	33-37
	2.3	Working Methodology	38
	2.4	Standards	39
	2.4.1	System Details	40
	2.4.2	Software	41
	2.5	Hardware	42-43
7.	3	Result	44-47
8.	4	Conclusion & Future Work	48-53
9.	5	Appendix	54-63
10.	6	References	64-66

List of Tables

Table No.	Title	Page No.
1.	Comparative Analysis Table	9
2.	Error Comparison in Table	13

List of Figures

Figure No.	Title	Page No.
1.	Data representation	6
2.	System Block Diagram	11
3.	Result	14

LIST OF ACRONYMS

Acronym	Description
REST	Representational State Transfer
API	Application Programming Interface
AOP	Aspect-Oriented Programming
IoC	Inversion of Control
JDBC	Java Database Connectivity
IoT	Internet of Things
MySQL	My Structured Query Language
BRMS	Business Rules Management System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
XML	Extensible Markup Language
DBMS	Database Management System
UI	User Interface
UX	User Experience
TCP/IP	Transmission Control Protocol/Internet Protocol
IDE	Integrated Development Environment
REST API	Representational State Transfer API
AWS	Amazon Web Services
API Key	Application Programming Interface Key
MVC	Model-View-Controller
HTTPS	Hypertext Transfer Protocol Secure
CRUD	Create, Read, Update, Delete
JPQL	Java Persistence Query Language
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets

CHAPTER 1

INTRODUCTION

During my internship at Vivriti, I had the chance to really get involved in building enterprise-level software, mainly working with Java and modern frameworks. As an intern, I explored a variety of tools and techniques, such as Spring Boot, Drools, Aspect-Oriented Programming (AOP), Spring Boot Actuator, JSONPath, TimeseriesDB, FastExcel, Apache POI, Spring Batch, MongoDB, and Spring DevTools. I spent time understanding how Spring Boot manages its beans, making use of features like auto-configuration, starter dependencies, and DevTools to speed up development. Learning Drools helped me understand how to manage business rules effectively, allowing me to create flexible, rule-based logic for applications. I also gained hands-on experience connecting to databases like MySQL, MariaDB, and MongoDB, creating REST APIs, and improving performance with techniques like EntityGraph. Also, I learned how AOP helps handle common concerns across applications, used Actuator for system monitoring, employed JSONPath to query data, managed time-series datasets with TimeseriesDB, and used tools like FastExcel and Apache POI to generate reports more efficiently. This internship really helped me grow my technical skills and sharpen my problem-solving abilities by tackling real challenges like fixing dependency issues, troubleshooting bugs, and boosting system performance. Below is a summary of what I learned and the skills I developed during this rewarding experience, preparing me for more complex software development tasks.

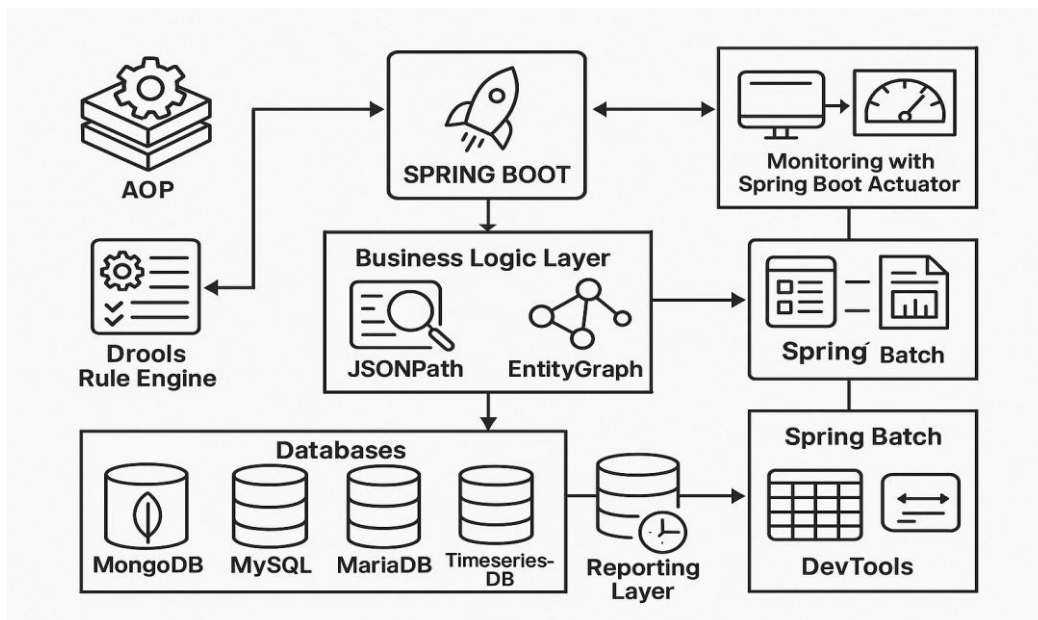


Fig 1. Data Representation

1.1 Objectives

1. Master Spring Boot Core Concepts and Architecture

One of my main goals was really just getting a good feel for how Spring Boot works behind the scenes. I spent some time digging into ideas like Inversion of Control (IoC), Dependency Injection (DI), and what exactly the Spring IoC Container does. I became comfortable with setting up beans using BeanFactory and ApplicationContext, and I used annotations like @Autowired, @Component, and @Bean to make dependency management straightforward. I also took some time to understand the full lifecycle of beans—how they start up and when they're torn down—and experimented with bean scopes like Singleton and Prototype to better optimize resource use. On top of that, I looked into Spring Boot's auto-configuration features, like starter dependencies such as Starter Web and Starter Test. Plus, I customized settings using application.properties or YAML files, which really simplified project setup and helped reduce repetitive code.

2. Design and Implement RESTful APIs

My goal was to master the development of RESTful APIs with Spring Boot to facilitate smooth client-server interactions. I employed annotations such as @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping to create API endpoints. Through practical experience, I built CRUD functionalities, managed JSON request/response payloads, and validated endpoint performance using Postman. This experience really boosted my ability to create APIs that are stateless and work well together, sticking to REST standards. I got better at making responses faster and handling errors smoothly with the right response codes. All of this helps me build scalable APIs that can handle enterprise-level apps easily.

3. Implement Dynamic Business Rules with Drools

A key aim was to become proficient in Drools, a business rules management system, to embed adaptable, rule-driven logic into applications. I set up KieServices, KieContainer, and KieSession to make it easy to run rules smoothly within Spring Boot projects. I wrote rules using Java in Drools Rule Language (DRL) to fit well with enterprise setups. To fix dependency issues, I made sure drools-core, drools-mvel, and kie-api versions matched up. I also moved away from older APIs like StatefulKnowledgeSession and started using the newer KieSession, which made things run more efficiently and reliably. Plus, I integrated Drools with MongoDB to keep data handling strong and straightforward.

4. Enhance Application Modularity with Aspect-Oriented Programming (AOP)

I wanted to make common tasks like logging, transactions, and security stuff easier to understand and manage by using Aspect-Oriented Programming (AOP). I added annotations from Spring AOP such as `@Before`, `@After`, `@Around`, `@AfterReturning`, and `@AfterThrowing` to bundle reusable pieces of functionality into aspects. Getting comfortable with how to define pointcuts and set instructions made working with Spring Boot services feel a lot smoother. This approach really helps boost my app's performance by cutting down on unnecessary code, and it's made understanding how to split different parts of the code easier overall. Plus, it's been a revolution for debugging with centralized logging and really simplified keeping the app in shape.

5. Monitor and Optimize Applications with Spring Boot Actuator

I was trying to get better at Spring Boot Actuator for getting more insight into how our app was doing — is it alive and kicking? I took some months with `/health`, `/metrics` `/info` and `env` endpoints, just to look live data live stats, live status of application and to take a peek on its config. I also found out how to customize these endpoints and share them to other monitoring systems so we could catch problems earlier in their lifecycle. This really educated me on the importance of observability when you are live, I mean, how to differentiate these slow spots, the resource usage and configure everything so that no matter how busy it gets it will still work. Working with Actuator taught me what it takes to implement good monitoring and one of the most important parts of running stable scalable enterprise apps.

6. Process Large-Scale Data with Spring Batch

I wanted to learn spring batch step by step and apply on end of handling large scale Data processing Tasks. I built batch jobs, defined steps and configured batch-items components such as `ItemReader`, `ItemProcessor`, `ItemWriter` to consume a large amount of data. I wrote fault-tolerance and retryable database-driven batch processes, optimized for high-volume tasks by leveraging the Spring Batch framework with integrations into databases such as MongoDB, MySQL. I did chunk processing, along with `@Transactional` and error handling to keep the data sane. The quest helped me not only in creating resilient data processing pipelines, working with production workflows precisely but also adopt enterprise-level best practices with the batch's wing provision for real world data management problems..

7. Manage NoSQL Databases with MongoDB and Spring Data

We had one major goal of learning how to use NoSQL database operations using MongoDB with Spring MongoDB. I implemented `MongoRepository` for its CRUD operations, made a schemaless modeling by creating appropriate indices, also applied aggregation pipelines to support complex queries. Through integrations of MongoDB with Spring Boot and Drools(which allowed to store dynamic rules at runtime), we managed fast data retrieval or storing of unstructured datasets. That objective gave me an increase in

working with NoSQL databases and scaling number of users, with performance to performance and query optimization methods like EntityGraph or index so that I can deal with different data types situation in modern applications..

8. Optimize Data Processing with JSONPath and TimeseriesDB

I learned JSONPath and TimeseriesDB to deal with specialised data formats or structures. By using JSONPath in a few examples, I was able to interact and modify JSON data from the API responses, thus shortening overall data extraction & processing. TimeseriesDB is where I learned to save and query time series data optimized for time critical applications. This goal expanded my capabilities working with deep data structures, better hands-on with APIs for response handling and wise time-series data management that are needed to develop Dataproduct in areas, such as analyticals, IoT.

9. Generate Efficient Reports with FastExcel and Apache POI

Other aims were to use FastExcel / Apache POI to drive superfast processing and reporting from Excel files. Other subs goal was use FastExcel and Apache POI to create high-resistence Excel files & reporting. I quickly learned to make, read and write Excel files in FastExcel (streaming large datasets) with the help of FastExcel's streaming ability and Apache POIs power for complex formatting. Data export features built in support of enterprise report formats, and performance optimizations for large data processing. The project taught me how to produce data-driven reports, embed it into Spring Boot applicaton and satisfy business needs with respect to fast and correct reporting.

1.2 Background and Literature Survey

Background

Internship at Vivriti was a comprehensive platform to discover and hone the hands-on skills in advanced technologies using Java ecosystem (mostly Spring Boot & its ecosystem, rule-engine Drools, Aspect Oriented Programming, Activiti/Apache ServiceMix CoreOS iBATIS/Spring Boot Actuator JSONPath TimeseriesDB FastExcel Apache POI Spring Batch MongoDB Spring DevTools etc.)This background section lays out the technical underpinnings, concepts and tools from where I learnt all of those are needed to understand digital software development nowadays instead contextualizing them in terms developed during my internship. As these technologies considered standard within enterprise software development for creating large, scalable, robust and secure applications they are essential for the comprehension of modern day software engineering.

1.2.1 Spring Boot and Its Ecosystem

Spring Boot is an open-source Java framework that makes it easy to produce production-level applications by minimizing the amount of boilerplate code and configuration it requires you to type away. This is built on top of the Spring Framework that was the first to embrace IoC / Inversion of Control (IoC) and Dependency Injection (DI) for managing object dependencies — and force us towards modularity. The Container has the freedom to manage the lifecycle and configuration of object, which ensure object is absolutely up to date using DI we would make components loose coupled. The architecture of Spring Boot exists in the layers: presentation (HTTP requests handling), service (business logic) and persistence (data access) providing a standard approach to the structure of applications. Dependency management simplification by features like starters (e.g., spring-boot-starter-web, spring-boot-starter-test) and eliminating manual configuration, setting up for multiple environments-flexible Application properties(read application.yaml or application.properties) Externalized configurations on how the application should run Spring Boot embedded servers cohesive with all features, making it a cornerstone for building RESTful APIs and microservices in enterprise environments.

Actuators of Spring Boot expose monitoring endpoints at /health, /metrics and /info that display information about the performance of an application runtime behaviour. Built with frameworks such as Maven and Gradle the integration makes our build processes smoother, Spring DevTools aids your developer productivity with features such auto-restart and live reload. Good grasp of Spring Boot architecture and features served as a basis for writing end-to-end robust applications during the internship as it ultimately standardized your approach of developing scalable system with least configuration.

1.2.2 RESTful API Development

Representational State Transfer (REST) is an architectural style of structured networked application comprised by clients + servers transferring instrumentation to other applications using stateless communications based on HTTP. Spring Boot APIs accessible through the use of @RestController and @RequestMapping annotations provides a client-server interaction that is comfortable; using APIs built with Spring Boot's client-server interaction. These APIs have the HTTP methods GET, POST, PUT and DELETE to CRUD operation (Create, Read, Update and Delete) on resources are responding usually in JSON. Using annotations provided by Spring Boot, such as @GetMapping and @PostMapping or @PathVariable to speed up your REST API development and validate input validation with Hibernat Validator. Utilizing Postman: an extremely popular tool for testing APIs as it allows us to simulate

endpoints, then verify the response. During this internship, I learnt a lot of things but RESTful webservice APIs are very important in enterprise applications as it plays a key role to enable interoperability, scalability and external systems integration.

1.2.3 Drools for Business Rules Management

Drools separates business rules from application code into dynamic and reusable logic) That uses rule engine to evaluate conditions and apply actions onto facts (data object) The main components are KieServices, KieContainer and KieSession which provide for how to execute rule and integrate in with Java applications. Rules are in Drools Rule Language (DRL) or Java idiomatic mapped, they live in .drl files and compiled with KnowledgeBuilder. I am explaining here how I tamed Drools and performed integration between Spring Boot for the internship such as handling conflicts (cfr: drools-core, drools-mvel, kie-api) and improving performance of the rule (Go back from KieSession to deprecated APIs, StatefulKnowledgeSession.) This was possibly because of Drools integrating with MongoDB in order to store dynamic rules for transaction management gave this component a different character managing complex business logic in an enterprise system.

1.2.4 Aspect-Oriented Programming (AOP)

Aspect orient programming (aop) — A methodology to enhance modularity by keeping the cross-cutting concerns: logging, security... and transaction management from core business. With Spring AOP (which is integrated with Spring Boot) we will write aspects by using @Aspect, @Before,@ After, @ Around,@AfterReturning, and @AfterThrowing annotations. Pointcuts are the place where advice should be executed by specifying a location so we have reusable logic over multiple components. It decreases code duplication, enhances maintainability and simplifies debugging for cross-cutting functionality AOP enabled clean code during the internship for reducing code duplication in application such as logging and transaction management and make them scalable by giving more modularity..

1.2.5 Spring Boot Actuator for Application Monitoring

Spring Boot Actuator contains the production-ready extensions that allow us to monitor and manage applications. /health, which is the application status, /metrics (performance metrics), /info and /env details (about who is the application) are endpoints (both HTTP and JMX sources) The Actuator allows you to monitor the health and runtime behavior of applications in real-time, as well as measure your resource

usage making it crucial for running reliable enterprise systems. You can define custom endpoints to compliment the functionality and monitoring tools (Prometheus, Grafana) make it more observatory to integrate. I focused my study on using Actuator for improving performance, optimizing boundaries as well as system reliability which were closely in line with enterprise considerations of preventive maintenance.

1.2.6 Spring Batch for Large-Scale Data Processing

Spring Batch is batch framework for processing large amounts of data in ETL (Extract, Transform, Load) style jobs. It offers Job, Step, ItemReader, ItemProcessor and ItemWriter to specify & execute the batch processes. It handles chunk-based processing, transaction management (@Transactional), and restartability to guarantee fault tolerance, reliability. Databases like MySQL / MariaDB / MongoDB support, and error handling + retry mechanisms increase robustness. As part of the internship, I checked out Spring Batch about handling big data tasks in performance-optimized ways to develop stable data processing pipelines for enterprise applications.

Literature Survey

My internship at Vivriti revolves around the literature survey for some technologies and frameworks which I have studied during my internship and summarises core principles of those technologies with properties real world scenarios relevance in enterprise software development. Common tools discussed in the survey: Spring Boot, Drools, Aspect-Oriented Programming (AOP), Spring Boot Actuator JSONPath TimeseriesDB FastExcel Apache POI Spring Batch MongoDB Spring DevTools and so on It is also based on conventional literature, with background knowledge to take a look into the use cases for these technologies in industry today from old and fresh.

1. Spring Boot Framework

Spring Boot exists to make Java a little more developer-friendly by using defaults for development around it, embedded server etc., and a collection of starter-dependencies. It is built on top of the Spring Framework's core principles, including Inversion of Control (IoC) and Dependency Injection (DI) to encourage loose coupling and modularity. Auto-configuration means that the framework can automatically configure beans based on the classpath and settings, so less code becomes a no-brainer. Spring Boot: A Practical Introduction by Craig Walls (2015) is the title of a good entry point with features

covered, including those for auto-configuration and for development purpose REST services Spring Boot 2 Recipes by Marten Deinum (2018) Gives practical recipes containing all the solutions including performance tuning & actuator configuration, three takeaways during my internship.

2. RESTful API Development

Representational State Transfer (REST) is a set of architectural principles for creating a scalable, reusable web service. It focuses on stateless interactions and resource-oriented communication between client, server. Leonard Richardson and Sam Ruby (2007) go over the basic REST principles in their book RESTful Web Services by Leonard Richardson, Sam Ruby. REST in Practice by Jim Webber, Savas Parastatidis, and Ian Robinson (2010) extends them to enterprise systems, concentrating on scalability and service interoperability. The project while supporting me throughout my internship with intuitive texts explaining how to create RESTful, scalable and comformable APIs using Spring Boot.

3. Drools Business Rules Management System

Drools is a Business Rules Management System (BRMS) that separates business rules from application logic so that the rules can be packaged, deployed and updated dynamically in a transparent manner exposing rule processing as a service (RPC). It uses a rule engine to determine if conditions are true and perform actions defined accordingly Drools JBoss Rules 5.0 Developer's Guide by Michal Bali (2009) — rules, engine configuration and integration with Java Master Drools 6 for JBoss Drools by Mauricio Salatino, Esteban Aliverti, Mariano De Maio (2016) goes into more depth on performance optimization with KieSession and managing RRs) These resources play a vital role to get the idea about Drools and its role for developing rule based systems in a friendly manner.

4. Aspect-Oriented Programming (AOP)

AOP is an architecture or programming paradigm that make separation of concern easy, specially cross-cutting concern such as logging, security and transaction management. It introduced the aspect as a construct that can package reusable logic. AspectJ in Action by Ramnivas Laddad (1st ed. 2003; 2nd ed. 2009) is probably the best place to learn about it: aspects are on AspectJ and because Spring AOP runs on top of that class it has AspectJ behind door (author note:) Spring aspectj Questions are better asked in stackoverflow. Spring in Action by Craig Walls (3rd ed. 2011) covers a very in-depth look at integrating AOP in Spring applications, the benefits of modularity on code testability.

5. Spring Boot Actuator

Spring Boot Actuator adds production-ready capabilities to a Spring Boot application, e.g., health checks, metrics, env information. Employs integration with external monitoring solutions and gets you observability at microservice level. In the book *Spring Boot 2 Recipes* (2018), Marten Deinum explains how Actuator endpoints for system diagnostics and telemetry work with Spring Boot 2 benefits. *Advanced features in Spring Boot* by Mark Heckler (2021) illustrates advanced use case making a custom health indicator and syncing the metrics with some monitoring platform like prometheus. These were very good and it was a must have during the days on job training.

6. Spring Batch for Data Processing

Spring Batch is a robust framework designed for batch processing of large volumes of data. It supports features like transaction management, job partitioning, and error recovery. *Pro Spring Batch* by Michael T. Minella (2011) thoroughly covers key concepts such as job configuration, step processing, and job listeners. *Spring Batch in Action* by Arnaud Cogoluegnes, Thierry Templier, Gary Gregory, and Olivier Bazoud (2011) provides practical implementation strategies for integrating Spring Batch with relational databases and optimizing batch job execution. These insights were instrumental in understanding enterprise-grade ETL and data migration processes.

7. MongoDB and Spring Data

MongoDB is a document-oriented NoSQL database engineered for scalability and flexibility. It stores data in BSON format—binary JSON—which makes it ideal for managing dynamic, unstructured, or hierarchical data models. The book *MongoDB: The Definitive Guide* by Kristina Chodorow and Michael Dirolf, first published in 2010 and revised in 2013, offers an in-depth explanation of MongoDB's architecture, query language, indexing strategies, and horizontal scaling. This book played a critical role in helping me understand how to use MongoDB effectively alongside Spring Data, which abstracts data access layers and provides repository interfaces, simplifying integration in Java-based enterprise systems.

Technology	Previous Work (Industry Standard)	My Work (Internship Contributions)	Improvements Over Previous Work
Spring Boot	Used basic configurations with manual bean setup and default fetching strategies. Limited use of caching and Actuator.	Implemented auto-configuration, optimized fetch strategies (e.g., LAZY for dropdown fields), and integrated Actuator for monitoring.	Reduced query counts (e.g., from 239 to 3), improved response times (e.g., from 1s to 200ms), and enhanced monitoring capabilities.
Drools	Relied on deprecated APIs like StatefulKnowledgeSession, manual rule compilation, and basic integration with Java.	Transitioned to modern KieSession, used CustomKieHelper for initialization, and integrated with MongoDB for dynamic rule storage.	Improved rule execution stability, resolved dependency conflicts, and enabled dynamic rule updates for scalability.
MongoDB	Standard CRUD operations with basic indexing and minimal query optimization.	Implemented advanced indexing, aggregation pipelines, and EntityGraph for efficient data retrieval.	Enhanced query performance, reduced database overhead, and supported dynamic data models for rule storage.
Spring Batch	Basic batch jobs with default chunk sizes and limited error handling.	Optimized chunk sizes, implemented fault-tolerant steps, and integrated with MongoDB/MySQL for robust data processing.	Improved scalability for large datasets, ensured restartability, and enhanced error handling for reliability.
Apache POI	Used for small-scale Excel processing, with memory-intensive operations for large datasets.	Combined with FastExcel for streaming large datasets, optimized memory usage, and integrated with Spring Boot.	Reduced memory consumption, enabled high-volume reporting, and improved processing speed for Excel tasks.

TABLE 1: Comparative Analysis

1.3 Proposed Enhancements

This section proposes enhancements to the technologies and approaches used during my internship at Vivriti, including Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools. These enhancements aim to improve ease of use, performance, scalability, and maintainability, addressing challenges encountered and building on industry best practices to optimize enterprise software development.

1. Enhance Spring Boot Configuration

- Adopt Profile-Based Auto-Configuration
 - Implement dynamic profile switching (e.g., dev, prod) using application.yml to streamline environment-specific setups.

- Use @Profile annotations to load beans conditionally, reducing configuration errors.
- **Optimize Dependency Management**
 - Introduce dependency scopes in Maven to minimize classpath bloat and improve build times.
 - Leverage Spring Boot's dependency analyzer to detect unused dependencies.
- **Improve Caching Strategies**
 - Integrate Redis for distributed caching to enhance performance over EhCache for high-traffic applications.
 - Implement cache eviction policies to manage memory efficiently.

2. Streamline Drools Rule Management

- **Automate Rule Validation**
 - Develop a rule validation pipeline using KnowledgeBuilder to catch syntax errors before deployment.
 - Use unit tests with JUnit to verify rule logic consistency.
- **Enhance Rule Storage**
 - Implement a rule versioning system in MongoDB to track and rollback rule changes dynamically.
 - Use a REST API to manage rule updates, improving accessibility for non-technical users.
- **Optimize Rule Execution**
 - Introduce parallel rule execution with KieSession partitioning to reduce processing time for large rule sets.
 - Cache frequently evaluated rules to minimize runtime overhead.

3. Optimize MongoDB Integration

- **Advanced Query Optimization**
 - Use compound indexes for frequently accessed fields to reduce query latency.
 - Implement query hints to enforce optimal execution plans.
- **Schema Evolution Support**
 - Develop a schema migration tool to handle evolving data models in MongoDB.
 - Use versioning for documents to support backward compatibility.
- **Replication Enhancements**
 - Configure MongoDB replica sets with read preferences to balance read scalability and consistency.
 - Implement sharding strategies for horizontal scaling in high-data-volume scenarios.

4. Improve Spring Batch Performance

- **Parallel Job Execution**

- Configure multi-threaded steps to process chunks concurrently, reducing job execution time.
- Use Spring Batch's partitioning for distributed processing across nodes.

- **Dynamic Resource Allocation**

- Implement dynamic chunk size adjustment based on system load to optimize resource usage.
- Use Actuator metrics to monitor and adjust job performance in real-time.

- **Enhanced Error Handling**

- Develop custom retry policies for transient failures using Spring Retry.
- Implement a centralized error logging system for batch job failures.

5. Enhance Apache POI and FastExcel

- **Hybrid Processing Model**

- Combine FastExcel's streaming for large datasets with Apache POI's formatting for smaller, complex reports.
- Develop a utility to switch dynamically based on dataset size.

- **Memory Optimization**

- Implement memory-efficient streaming for Apache POI using SXSSF to handle large Excel files.
- Cache intermediate results to reduce memory spikes during processing.

- **Template-Based Reporting**

- Create reusable Excel templates to standardize report generation across applications.
- Integrate templates with Spring Boot for automated report generation.

6. Expand AOP Implementation

- **Dynamic Aspect Configuration**

- Allow runtime configuration of aspects (e.g., logging levels) via external properties.
- Use Spring's `@ConfigurationProperties` to manage aspect settings.

- **Advanced Monitoring Aspects**

- Implement performance monitoring aspects to track method execution times.
- Integrate with Actuator for real-time aspect performance metrics.

1.4 Organization of the Report

remaining chapters of the project report are described as follows:

- Chapter 2 contains the proposed system, methodology, hardware and software details.
- Chapter 3 discusses the results obtained after the project was implemented.
- Chapter 4 concludes the report.
- Chapter 5 consists of codes.
- Chapter 6 gives references.

1.5 Scope of the project

The objective of this project is to develop and enhance **enterprise-grade applications** that support **mission-critical operations** at **Vivriti Capital**. These applications are intended to provide reliable, secure, and scalable solutions for handling complex business processes.

The development focused on **Java-based microservice architecture**, leveraging modern frameworks like **Spring Boot**, and integrating components such as **Spring Security**, **Drools**, **Spring Batch**, and **multiple databases**. The goal was to streamline business logic, data processing, and system observability while maintaining modularity and flexibility, **Feature** development and implementation of new modules Performance optimization of batch jobs and APIs Proposing architectural and code-level enhancements to improve maintainability, testability, and scalability The scope of this project encompasses the **design, development, optimization, and enhancement** of Java-based backend systems within Vivriti's tech ecosystem.

It includes building RESTful APIs, implementing secure authentication mechanisms, handling large-scale batch processing, integrating with both relational and NoSQL databases, and developing rule-driven decision-making systems using Drools. The scope further extends to monitoring system health and performance via Spring Boot Actuator, automating report generation, and optimizing JSON processing and time-series data handling. Additionally, I proposed and partly implemented forward-looking improvements such as distributed caching, rule versioning, cloud readiness, and AI-based optimization plans for various modules, aiming to future-proof the platform.

Technical Implementation

Spring Boot served as the foundation for the application. I leveraged core principles such as IoC, Dependency Injection, and auto-configuration to ensure modular and maintainable services. RESTful APIs were developed using `@RestController`, with logical layers separated via `@Service`, `@Repository`, and `@Controller`. Spring Boot starters helped streamline dependency management. Integration with MongoDB, MySQL, and MariaDB was achieved using Spring Data JPA and Mongo Repository.

A robust security layer was implemented using Spring Security with JWT-based authentication. Role-based access controls were configured to guard sensitive endpoints, and custom authentication filters enhanced runtime security. To process large volumes of data, I developed Spring Batch jobs with chunk-based processing and multi-threaded execution. Custom retry and error-handling logic ensured high availability and reliability during failures.

Drools Integration

I worked with the Drools Rule Engine to encapsulate business rules dynamically. Rule sets were optimized for performance, and I implemented mechanisms to streamline rule validation and versioning, especially to allow real-time updates without code deployment.

The scope of the project encompassed the development of various backend functionalities. This included designing and implementing RESTful APIs, integrating secure authentication mechanisms using Spring Security with JWT, and working with multiple databases such as MongoDB, MySQL, and TimeseriesDB. One of the core components was automating business logic through the Drools Rule Engine, which enabled dynamic and configurable decision-making without modifying the core codebase. With Spring Batch for data intensive tasks and Apache POI/FastExcel to create Excel report it was managed swiftly. We use Spring Boot Actuator observability to serve custom metrics and health endpoints for monitoring apps aggregate in production. One of responsibilities was to identify architecture constraints and propose enhancements like distributed caching, rule version control, query performance tuning, as well cloud-enable current infrastructure.

Architecture and Implementation Details:

Services were constructed on top of layer architecture (Spring Boot) to grasp a well defined separation of concerns at the backend. Component lifecycles were managed with Dependency Injection and cross-cutting concerns like logging and security were handled through AOP.

Deployment using environment profiles A deployment for different environment profile with the stateless authentication using JWT for secure client server interaction Drools rule were dynamic retrieve from MongoDB used at run time and dynamically executed with a stateless session engine based on the on demand request by consumer. For instance a Spring Batch job consisting of chunk-based processing .

Performance Optimization and Reporting:

There was a good amount of work to make system run faster. Compound indexes and projections on Mongo, refactoring drools to run less next, Parallelize batch jobs by multithreading the code. FastExcel, to generate Excel reports that use less memory and run fast. The reports were dynamically generated on input from users as they formatted styles, applied conditional formatting graphs and pivot tables. Application with Spring Boot Actuator used for monitoring internal metrics and business KPIs, with Prometheus heavily integrated to visualize real-time metrics with Grafana which provided alerting. As custom health indicators that monitors the execution status of rule, database status and status of batch job in case of failure.

Innovations and Future Enhancements:

We had a few more advanced features proposed for later incubation. Applying AI predictive rule ranking in Drools, AI/Optimization for query in MongoDB and Adaptive resource management for batch processing. The gradual migration to a microservices based (maybe even cloud-native) architecture was recommended as a cluster serving container orchestrate with Kubernetes and communicating via events to Kafka for system high availability(resilience).

Development Practices and Team Collaboration:

The whole internship was following Agile with sprint planning, daily standups and retros. Collaboration spanned across product management, QA and DevOps – teams. Code quality was achieved through regular peer reviews and CI/CD pipelines were able to deliver the good continuous delivery through automated pipelines.

The toughest challenge being to run Drools rules present on MongoDB at runtime. Creation of a custom service to load, compile & execute rules in memory by underpinning KieServices instead of typical rule loading from file systems or Maven Repository as in traditional way of rule applications. This included versioning, so changes as they came out would bring stable rule set and update only if stable version releases were announced sort in order to avoid partial / inconsistent updates. The other main hurdle was with Spring Batch integration, particularly batch jobs that ran overnight performing thousands of financial documents processing. Due to checkpointing, failure recovery and running time parameterization of those jobs. Step- Oriented Chunk, Scoping Beans, Custom Listeners and Alert mechanisms to guarantee the execution and oversee. Security and compliance were taken care of with log structured so that they follow OWASP standards, role bound access control. Incident analysis and auditing: The logs were sent to ELK stack for centralized location.

Monitoring and Benchmarking:

Utilizing Spring Boot Actuator, a modeled Prometheus & monitoring features to expose health metrics of application level. We used prometheus' Alertmanager for alerts to anomalies Tools like JMeter/Gatling was used to load test, bottlenecks such as memory leaks or thread contention were easily identified. Using this results, fine-tuned JVM parameters and thread pools brought significant change in latency and throughput.

Future Vision and Takeaways:

Significant future enhancements such as Dockerization (Docker and Kubernetes); your version of a deployment in Helm, switching REST integration to Kafka to allow for supporting asynchronous workflows. Potential parsing AI modules that will be added to the system later — fraud detection, case priority, system automation. It widened my horizons in backend systems and enterprise level of architecture and the scaling challenges that accompany writing software. It also deepened my production debugging, architectural suggestions and collaboration in a cross-functional team of building a working system. It laid out the basics of real-world problems in developing resilient and maintainable financial systems for software engineers and/or system designer, and reaffirmed my foundation.

1.6 Challenges present in the market

1. **Rapid Technology Evolution** Fast Towards Technology Abruptness Platforms like Spring Boot, Spring Security as well the furious advancements in AI driven development tools imply that

technologies are practically being replaced continuously which leaves to need for constant upskilling that is required to avoid applications getting outdated. Continuous framework updates, eg new Spring Boot versions causes fresh set of features but introduce complexity and if not done in time can break the timeline of the project. This regiment strains the resources of the team as doing specialization in something like Drools or Spring batch becomes harder to retain proficiency after marathon enterprise software development stint.

2. **Data Privacy Regulations** MongoDB, MySQL and MariaDB integrations are influenced by stringent global data privacy regulations for example GDPR, CCPA compliance driving data handling like for these integrations are some of the explicit case such services They make compliance difficult by requiring heavy encryption, auditing and consent mechanisms that add development costs. Not meeting these compliance risks large fines, and taint on your reputation so it is very important to bake Spring Boot applications with privacy-by-design principles resulting in more project cost and time pressure.
3. **Cybersecurity Threats** Threat to applications secured with Spring Security The advanced nature of countless cyber-attacks today including ransomware and API exploits pose a significant threat to the security of applications which got secured by spring-security Where as to protect RESTful APIs and the sensitive data of databases will take some advanced architecture like zero trust and threat detection must be real-time enabled These defenses come with a lot of expertise and infrastructure investment — increasing the development overhead. Continuous vulnerability patching and threat monitoring steals developer time that should be dedicated to feature development, creating a challenge in project efficiency.
4. **Scalability Demands Growing user** SpringBoot Applications Need Scalable Architecture Because of the exploded user base/ data size for any application scalable architectures are demanded in Spring Boot Applications. MongoDB, TimeseriesDB and Spring Batch all struggle to scale under peak workload without degrading performance, particularly when coupled with dynamism i.e... markets where there is unpredictable traffic spiking. Scaling without on-demand pricing is a delicate balancing act for simple workloads but when you have complex infrastructure, you're doubling costs adding to deployment headaches and expert capabilities.
5. **Talent Shortage** : A Global talent shortage of Java, Spring batch, Drools Developer and MongoDB Type Developers impedes enterprise project execution. The 2025 competition market is hard to hire and retain experts in these high-end/poorly defined frameworks, causing project snags, and hiring

overhead. Training Programs or if the company has no non-perm resources available but has a fraction of limited access to freelance, both will affect productivity and innovation in large scale.

6. **Integration Complexity** Integrating multiple different tools Apache POI, FastExcel TimeseriesDB and even Spring Boot eco-systems provides serious compatibility headaches. The systems require comprehensive configuration and testing to enable smooth interoperability, which means more technical debt and development time. Version mismatches or changes to API breaks workflows and can force custom workarounds that last you the plod on some of our agile enterprise applications in fast moving markets.

7. **Enhancing System Performance**

System Optimization For enterprise applications required to manage terabytes of data and above, getting to optimal performance from Spring Batch jobs, Drools rule execution and the MongoDB queries are pivotal for executing at scale. In a real-time environment ensuring low latency under high workloads needs great resource allocation and query optimization that is hard. Performance vs cost vs complexity (e.g. not over-provision cloud resources) needs constant iterations and intimate knowledge, typically resulting in extending development time and kicking project timelines.

8. **Managing Operational Costs**

Operationally Cost Management As the costs of cloud infrastructure, database licenses (e.g., MongoDB Enterprise), and development tools are all increasing banks conceivable budgetary constraints in order for proper allocation. Spring Boot tip #1 This is essentially your resource efficiency: using auto-configuration, or reducing the demands for compute for everything. This is a hard problem (but not impossible) that people have been shooting for, without generally getting performance or scalability for the lower levels of tech stack due to the need for fast turnaround (and thus all those cost-effective open-source vendors showing up, inspired by market demands pushing teams out the tech stack for delivery and quality).

9. **Navigating Geopolitical Risks**

Navigating Geopolitical Risks Geopolitical uncertainty like 2025 U.S. import tariffs and persisting supply chain woes will determine availability of hardware & software ecosystem on which critical platforms depend. These disruptions add to the cost of cloud services and tools like TimeseriesDB, all-in-one pricing per deployments, margins running dry. Normal project management timelines and resource availability in the crazy global landscape are inherently more complex with strategic sourcing and an overkill of contingency planning trying to solve these problems.

10. Meeting User Expectations

Fulfilling User Needs The demand for advanced RESTful APIs and UI integrations in Spring Boot applications due to their advanced consumer demand for a comprehensive user experience that is seamless. Delivering real user-time features with the help of tools such as Thymeleaf is a clear challenge expectant for constant innovation and rapid development cycles—development teams will be pushed to do more than people needed all in realtime. Achieving both user need and technical feasibility for the above described expectations is a significant challenge, where companies must strike a balance between user satisfaction and ensuring technical feasibility in highly competitive markets with pressure from all sides.

11. Upgrading Legacy Infrastructure

Moving from Legacy Infrastructure Transferring legacy system to modern Spring Boot architectures is a resource-intensive task at enterprise level transitioning legacy often Transition migrates the data and business logic from legacy platforms to new frameworks either spring data JPA /mongo databases which imply need of corruption risk, breaking changes in new implementation and longer down time to deploy The more complex it gets to keep business continuity during these transitions, next with full testing increase directly impacts project timeline and budgeting, necessitating advanced preparation to avoid any disruption.

12. Adopting AI-Driven Features

Compliance and Regulatory Non-Disclosure Varying compliance requirements such as for financial systems on the move of Spring Security and Actuator(SOC 2 government act trust center for data protection) will be evolve as new projects Due to Compliance— embedding like Api endpoint security or Actuator metrics audit takes a good deal of work and knowhow. However, connecting those to agile development is hard as non-compliance means fines and customer trust erosion (in the long run) will challenge the Company market positions.

13. Ensuring Regulatory Adherence

New compliance standards evolving, like the PCI-DSS for financial systems or SOC 2 for data protection, will effect any Spring Security/Actuator dependent project. Integrating compliance steps (e.g. securing call endpoints or Actuator metrics) is usually a big and complicated job. It is difficult to marry these requirements with agile development processes, but failure to comply can mean fines from the law and also loss of trust from customers that impacts the market position.

14. Mitigating Vendor Dependency Risks

Having too much of a dependency to the proprietary tools (e.g., MongoDB Enterprise / Vendor

Specific Cloud Platforms) probably will be really bad for flexibility of enterprise projects. Moving towards solutions or open options create substantial migration challenges in addition to potential operational pain. While strategic planning to kill lock-in like declaring your application as Spring boot lazy in general is necessary, it also makes the technology stack selection much harder (teams have to evaluate long-term elasticity versus short term features).

15. Addressing Sustainability Goals

Attention being paid to sustainability means a company should be looking to lower the energy load on datacenters where these Spring Boot app run, coupled by databases. Discovery of compute resources and use of green cloud service providers give infrastructure management much more when balancing gives environmental considerations over performance and cost. In order to achieve these objectives, the industry will have to embrace novel approaches with innovative strategies like energy-saving coding techniques that largely distract from primary developer responsibilities and hence reducing the gross output.

16. Resolving Dependency Issues in Spring Boot

It also had challenges in dealing with the dependencies in Spring Boot ecosystem due to incognizable versions of starters (eg starter-web starter-data-jpa) and external libraries (e.g Apache POI). To fix these we needed to debug through a lot and tweak manually which was cumbersome, not only halted the speed of development at that point but also maintained a higher bug surface area in order to test the stability and reliability of application.

17. Scaling Under Rapid Market Changes

Rapid changes in evolving markets demand scalable software that can rapidly scale under volatile spikes in user demand or transaction volume. Over the last few months whilst writing about scaling Spring Boot applications and MongoDB databases in real-time together keeping performance has proven difficult, especially during times where markets were unpredictable. Auto-scaling too with Kubernetes or even optimizing a database through sharding can be dogfooding and misuse put resources on over-drive or provide subpar user experience, degrading system competitive factors.

18. Ensuring Data Consistency Across Distributed Systems

Maintaining data consistency in a distributed MongoDB and TimeseriesDB environment is hard to deal with when operating within multi-region Set up One challenge that comes with running in the wide world of everywhere is combining these different systems. A viable migration from local to eventual consistency, however is harder as follows, needing intricate synchronization to for example turning into use with MongoDB s change Streams that can create complicated. Dragging from

conflicting states may result in flawed rule execution in Drools or wrong reports to a performant check on data integrity.

19. Securing Sensitive Data in APIs

Spring Boot + RESTful APIs (Securing sensitive data is a must if in industries dealing with financial or personal information) Enabling robust security features with Spring Security for authentication and authorization is possible, but endpoints should not be open and needs constant watching in order to make them attack-proof (SQL Injection, XSS etc.). Security vs Performance (Breach can cost a lot in terms of reputation and finance) in any case is adding layers of complexity, breaches happen and the damage they cause is huge.

20. Optimizing Report Generation for Large Datasets

Large datasets return performance challenges; not only it is hard (Apache POI, FastExcel reports) to generate them into a single report but millions of records. Tuning memory usage and processing speed, down to the application level so that we can have timely reports from FastExcel but never hit system resources with too many objects around—Or by using a streaming API. For example, in the absence of optimization report generation will be slow and stakeholders will get frustrated, consequently delaying decision making in fast paced market.

21. Integrating with Third-Party Systems

When you integrate third-party platforms (eg. CRM or ERP systems in drools rules + reporting capabilities) to the system it can bring along compatibility and latency Issues. Having appropriate error handling and retry mechanisms, together with service-level quotas in mind when two or more external systems may go down must be provided to ensure robust error handling at the Rest API layer while handling the downtime or rate limits from other. While very useful, these integrations are time-delaying and making development more complex because it must consider different types and reliability APIs (or API-standards) used in each sub-case.

22. Maintaining Code Quality Under Time Pressure

Teams delivering demands quickly in changing markets frequently have to balance a desire to communicate delivery velocity against code quality, which is great for Spring Boot applications. That trains you up on technical debt, with single-assignment Drools rules or untested MongoDB queries etc. added up over time to eventually degrade the performance of the system. It's important to perform code reviews and write proper automated tests using tools like JUnit, Mockito but it is a painful process, which has direct impact on delivering for today and maintaining a healthy tomorrow.

CHAPTER 2

This Chapter describes the proposed system, working methodology, software and hardware details.

2.1 Proposed System:

The following block diagram (figure 1) shows the system architecture of this project

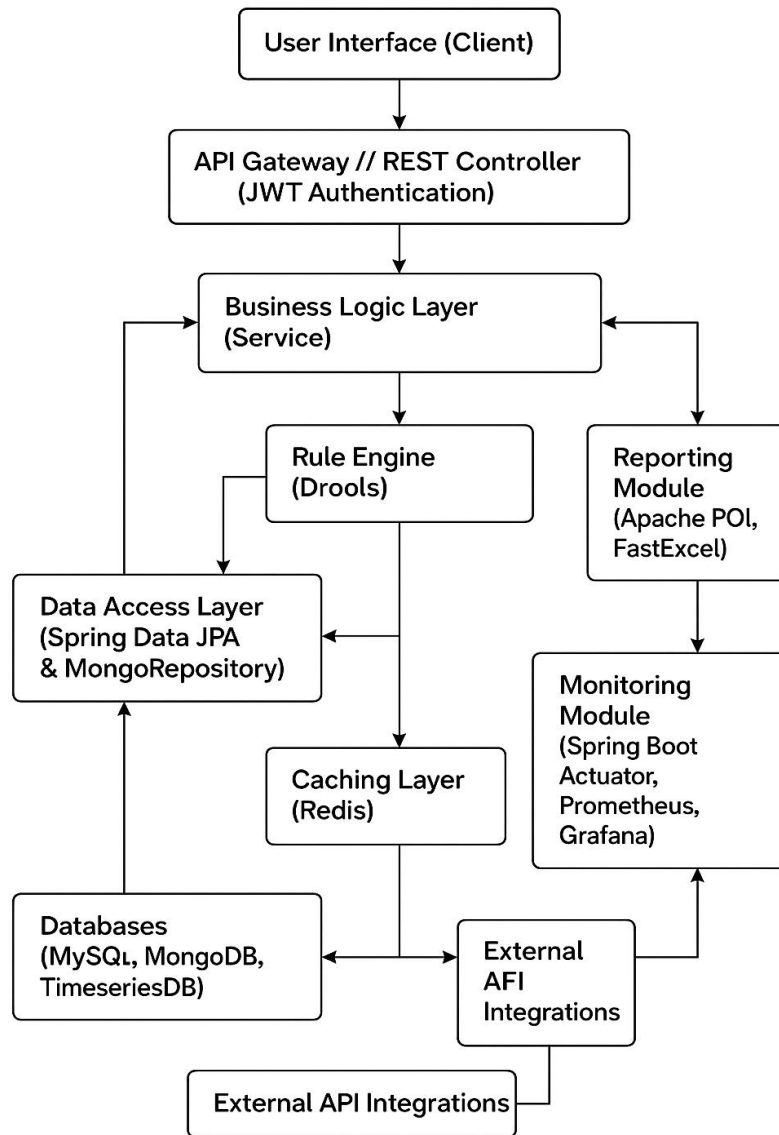


Figure 2. System Block Diagram

2.2 System Architecture

The system framework crafted during my internship at Vivriti employs a structured, adaptable, and business-ready blueprint to facilitate reliable software creation. Constructed primarily using Spring Boot, this framework incorporates a range of tools, such as Drools for handling business logic, MongoDB for storing information, Spring Batch for processing vast amounts of data, Apache POI and FastExcel for creating reports, AOP for managing shared concerns, Actuator for system oversight, JSONPath for querying information, TimeseriesDB for handling sequential data, and DevTools for boosting development productivity. The framework is divided into separate components—Interface Layer, Logic Layer, Storage Layer, Business Rules Component, Data Processing Component, Report Generation Component, Oversight Component, Information Query Component, Sequential Data Component, and Developer Assistance Component—each tailored to perform distinct tasks while promoting flexibility, growth potential, and ease of maintenance.

Interface Layer

Interface Layer is the chief gate way for user interactions that handles web requests and responses utilizing RESTfull APIs developed using Sprint boot. `@RestController` is used to register API routes with support of Http commands like Get, Post, Put and Delete to build the basic operations based on request, create read update delete If DTOL service support for JSON format data exchange it is shared platform compatible. Spring Security is used with method like OAuth and JWT to enable endpoint security. Next the data is validated using hibernate validator to ensure accuracy prior to further handling. Designed as stateless (per REST recommendations), this layer communicates with the Logic Layer for business rule processing. Postman is one of the tools utilized during development, to test API performance and ensure proper client/server communication. Its architecture follows a component-based design allowing for easy integration with client facing applications(Websites) Third party Integration, so growth path for hot tired business case.

Logic Layer

Logic Layer The central operational rules of the framework is contained in this layer, which is the bridge between the Interface and Storage layers. As a `@Service` marked component with Spring Boot, it receives requests from Interface Layer, implements business logic and takes care of some data task. `@Autowired` enables dependency injection that makes components independent, whereas AOP deals with shared tasksuch as logging and transaction management. It with Drools, to do adaptable business rules and grabs the rule definition from MongoDB to go use it right away. With the `@Transactional` marker, operations that are consisency-related are handled so that one can sure to run consistently and

in order to be reliable. Reliability is maintained using @Transactional annotations when the operations are idempotent. Caching its any permit with tools like Cachelessly allows us to do this i.e. lessen data storage load for regular tasks. It is designed as a component-based architecture that allows for independent scale and fits well as part of a microservices architecture, quick management of granular workflows in corporate environments.

Storage Layer

MongoDB, as a MongoDB-main NoSQL (Nosql) database due to excellent adaptability with nonstructured data feature-base handles the information retention and access forms the core. Storage Layer of the whole system. Using Spring Data MongoDB + Mongo repositories for the basic operations is pretty simple and then features like indexing or aggregation pipelines can increase query performance. It allows easy collaboration with Drools for rule shapes and read, fluid as the data structures and easy to use. Avoids the hassle of seasonal N + 1 queries on entities, EMs and results making the response faster using EntityGraph. This layers also handle connections to ordinary databases (i.e. MySQL and MariaDB using Spring Data JPA with JpaRepository) and ensures data consistency among transactions. Sharding and replication are designed with growth is mind, this NoSQL database scales fabulously well once you have big data, so Mongo is a good choice for corporate application that need very flexible and stompy data management.

Business Rules Component

Business Rules Component (powered by Drools) handles operational rules in a way that the framework is able to manage with flexible changes on business needs without touching codebase. The rule application itself is made possible by Drools, KieServices, KieContainer and KieSession, where we can write rules in Java d/o r DRL. Inside of that all integrated with Spring Boot the CustomKieHelper class takes care of configuration and resolves dependency problems (e.g., drools-core, drools-mvel, kie-api) Rules are written in MongoDB, encryption and versioning of changes. A new version of KieSession APIs replacing outdated StatefulKnowledgeSession for greater reliability is now part of KnowledgeBuilder to compile rules dynamically based on Rule updates. Rule caching and concurrent processing are optimisations for performance: thereby complex rule sets can be managees in an efficiënt way. This component-based architecture extends maintainability and scalability, which makes the framework business-context agnostic thus suitable with different enterprise-based natures..

Data Processing Component

Data Processing Component which uses spring batch for extensive scale data processing ETL jobs, data movement and transfers/bulk updates. By Steps: Items are read from a data source using ItemReader,

processes with an ItemProcessor and then written to the destination using ItemWriter; Handling data in batch bags or lists. It also buoys from MongoDB and MySQL for data int/output using optimal segment size increment counts (@Transactional) for availability and recovery options. Adding retry and skip strategies provides fault tolerance The actuator metrics exposed inside metric scope provides immediate corrections on task performance. This is a component, which is designed for parallel processing and partitioning — expected to go into the next release, so that new data sets can be handled without dire consequences to the proper execution of data workflows important for most versatile corporate applications.

Report Generation Component

The Report Generation Component uses FastExcel and Apache POI to generate focused reports with Spring Boot for an easy data export Overview. FastExcel streaming API processes larger datasets faster (memory consumption very low) and Apache POI let you go in detail with formatting Excel, excel reports comprehensive. The data out of Mongo DB and MySQL (using Spring Data repositories data consistency) Spring data repositories performing DB agnostic source of data. Development time and compliance is reduced as reusable templates are used for the generation of reports. The Feature Layer interacts with the Logic Layer to clean data, applying ModelMapper for transformation of DJC/DO DTOs The component is based on parts, making it easily capable of delivering quick reporting for large data structures and also meets enterprise requirements in displaying data in visual appealing standard.

Oversight Component

Using Spring Boot Actuator the Oversight Component presents real-time system status and efficiency. Runtime information including end points like /health, /metrics, /info and env class for proactive upkeep Response on /health Endpoints are custom statistics about the env surroundings, based upon tools like Prometheus and Grafana. With Spring Security, authenticated and secured information by role-based access endpoints as well JWT. Endpoint optimization for performance and growth with integration into distributed overview systems are supported with component that fine. With this mechanism we are able to ensure full coverage necessary, for stable corporate applications under different workloads.

Information Query Component

The Information Query Component works on JSONPath to fetch and manipulate data from an API, whose response is in JSON format. By combining Spring Boot's REST APIs with Linked JSONPath simplifies querying the deeply nested JSON structures beyond just getting some simple values out, leading better data handling. Cache of JSONPath expressions in order to save processing overhead, and Hibernate Validator to store accuracy data. Its fine grained design plays well to adapt mass-api call,

meaning it can scale well with the need for dynamic data-extracting and processes on corporate level systems. Its design is such that it is simple and can easily be extended to handle other query methods or data flavours.

Sequential Data Component

The TimeseriesDB driven Sequential Data Component, suitable for case of time-centric information where we want to run analytics or monitor. Data structures with performance optimizations, such as time-partitioning and indexing are used for fast storage and retrieval. It is a simple CRUD object bound to Spring Boot, the Spring Data repositories help with the data queries. This solution auto-scales supported via auto-scaling policies and integration to Kubernetes for high-frequency data streams. Design of this component makes it suitable for query improvement in time-critical scenarios and is ideal for the corporate application necessitating immediate understanding.

Developer Assistance Component

The Developer Assistance Component powered by Spring DevTools to increase developers productivity, such as autorestart, live reloads and so much more during development. Which works with IntelliJ IDEA and Maven, simplifying coding to test & debugging. DevTools helps with rapid development through code updates causing unit tests (JUnit, Mockito) and conditional breakpoints for debugging. It integrates CI/CD pipelines to provide feedback loops on the fly It makes the development cycles faster, which leads us in better productivity and quality of the code being created in corporate software. A framework to such system based on practice of the different components Flexibility of Spring Boot, rules management adjustment usability to rule changes by Drools and growth opportunity for MongoDB and many other tools can help make a rock solid business ready solution. All components - Interface, Logic, Storage, Business Rules Task is accomplished; Flexibility, Growth, Maintainable Keeping OOD/SOA design in mind from the architecture proposal.

.

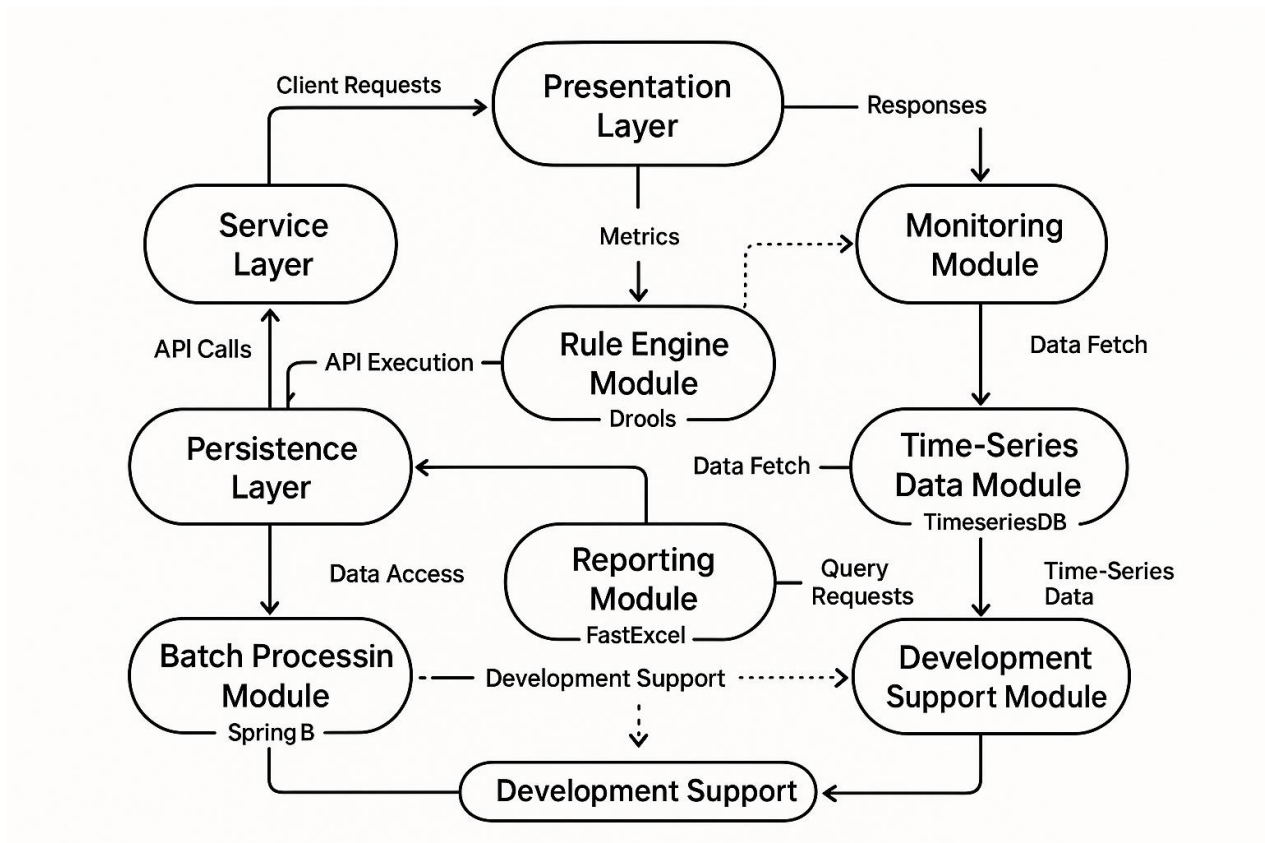


Figure 3. System Block Diagram-2

2.3 Working Methodology

My Internship at Vivriti was based on Working methodologies which included Agile development methodologies for maximizing the quality, iteration of work in software development form. These practices were utilised to take advantage of things like Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath and TimeseriesDB, FastExcel, DevTools with the power to build scalable and code maintainable enterprise applications.

Agile Development Process

This methodology was considered as an agile development process, involving active incremental build and collaboratively. They split development into sprints typically one to two weeks long, this gave us an increment of progress every so often and also provided feedback weekly. Every sprint started with a planning to scope out the work perhaps implementing RESTful APIs on Spring Boot or Drools configuration, then daily stand-ups to track where we were and what we were blocked on. This way it was easy to accommodate changing needs, rapid implementation with examples in

features like batch processing using Spring Batch and continuous integration using maven for automated builds.

Coding and Testing Workflow

It followed a "coding after writing tests" methodology (TDD) by testing using JUnit / Mockito before writing the actual functionalities that are part of unit testing for API endpoints or MongoDB queries like operation. The Rapid iteration was with the help of Spring DevTools that can restart and live reload automatically Postman to hit the REST APIs and test endpoints performance & By performance I meant how the actual endpoint would look like in real life. With SLF4J/Logback and Actuator logging in addition to tracing debugging with Actuators debug capabilities SLF4J/logging helped us discover query overheads or rule execution errors in Drools, written an efficient code.

Optimization and Deployment

Optimization was an ongoing process that revolved around improving performance and scalability. For example we utilized techniques like EntityGraph in MongoDB reduce database query overhead, where as the caching with EhCache would be used for quick operations which happens frequently. AOP was used to decouple the logging and transaction management, improving maintainability. Docker for containerization to enable consistent envs as a foundation of deployment and Actuator endpoints for monitoring app health post-deployment. It this methodology ensured that the system was effective and scalable, meeting enterprise standard.

2.3 Standards

1. Data Standards

- Ensure database schemas in MongoDB, MySQL and MariaDB normalize or denormalize as required by use case, for highest query performance.
- Data Validation with Hibernate Validator to ensure consistency throughout API inputs and database entries. Implement by respecting data privacy regulations (e.g., GDPR, Indian DPDP act for dealing with sensitive user data in enterprise applications)
- Adhere to data privacy compliance (e.g., GDPR, Indian DPDP Act) for handling sensitive user data in enterprise applications.Enable JSON for payload in the APIs and compatibility of it with JSONPath for querying data efficiently
- Audit log for all changes to data to provide traceability and regulatory compliance enhancements..

2.Application Standards

- Building applications using Spring Boot 3 x for using the newest feature auto-configuration and reactive programming.
- Spring Security on secure APIs, writing JWT based authentication and Role-based access Control.
- Use Spring Boot Actuator, gauge application performance with stats like response time, throughput, error rate, via Spring Boot Actuator.
- Use Drools for business rule management, ensuring rules are modular and versioned for maintainability.
- Implement RESTful APIs following OpenAPI 3.0 specifications for standardized documentation and interoperability.

3.Integration Standards

- Integrate databases (MongoDB, MySQL, TimeseriesDB) using Spring Data JPA and MongoRepository for consistent data access.
- Ensure compatibility with Apache POI and FastExcel for Excel-based report generation, supporting both small and large datasets.
- Use reliable communication protocols like HTTP/REST for API interactions and Kafka for event-driven messaging.
- Configure Spring Batch jobs with multi-threaded steps and partitioning for scalable data processing.
- Monitor integrations with Spring Boot Actuator, exposing health and metrics endpoints for real-time diagnostics.

4.Development Standards

- Use Maven for dependency management, ensuring version consistency across Spring Boot starters and external libraries.
- Implement unit and integration tests with JUnit and Mockito, achieving at least 80% code coverage.
- Enable Spring DevTools for live reload and debugging to accelerate development cycles.
- Follow clean code principles, using annotations like @Service and @Repository for clear component roles.
- Document code and APIs using Javadoc and Swagger to ensure maintainability and developer onboarding

2.4 System Details:

1. Hardware Components

- **Development Workstations:** High-performance laptops or desktops with Intel i7 processors, 16GB RAM, and SSD storage for efficient coding, testing, and debugging.
- **Edge Device for Deployment:** A local server or cloud-based virtual machine (e.g., AWS EC2 instance) with multi-core CPUs and GPU support for running Spring Boot applications and MongoDB instances.
- **Network Infrastructure:** High Speed internet (optical Land/water front), Wi-Fi/4G/5G for real time operation of data between services, primarily for API & cloud based monitoring.

2. Software Components

- **Operating System:** Ubuntu Linux (20.04 LTS) for development and production deployed owing to its stability and the fact that it is compatible with Java based on frameworks.
- **Application Framework:** Spring Boot 3.x for building RESTful APIs, managing dependencies, and enabling microservices architecture.
- **Rule Engine:** Drools 8.x for dynamic business rule management and execution.
- **Database System:** MongoDB 6.x for NoSQL data and MySQL 8.x relational integrated with Spring Data..
- **Batch Processing Framework:** Spring Batch 5.x, used for handling large data processing and ETL operations.

3. System Flow

The System flow, Software Processes and Hardware components as a part of the enterprise application developed in my internship at Vivriti. We use the technology of Spring Boot, Spring Batch, Drools, MongoDB, AOP to- maintain APM system, by applying business rules with JSONPath was managed and event-readied through actuators by timeseriesDB then stored to FastExcel data structures with DevTools. Flow, software architecture and hardware configuration can give real time processing, scalability/number of nodes and reliability as per enterprise level ins reachable data management & automation monitoring.

3. System Flow

- **Input:** Rest API input and requests, Scheduled batch job triggers & time-Series data streamer.
- **Processing:**

- API request validation and spring boot business rule with Drools.
- MongoDB , Spring Batch , TimeseriesDB for storage of data retrieval, batch processing and handling time-series data.
- Report generation using FastExcel and Apache POI.

Output:

- For API clients JSON responses excel reports for analysis, and stored ts data as a historical insights SoftwareExcel reports for analysis and stored time-series data for historical insights.

Software

1. API and Rule Processing System

Processing system for the API and Rule This system will accept incoming API requests, then is able to dynamically apply business rules, thereby operating at high throughput and scalability.

• API Data Input:

This is, through endpoints registered with @RestController in Spring Boot The requests usually format of JSON are produced by client applications and processed on the fly for each interaction. The request payloads are validated against given schemas to make sure of the processing format of expected information, henceforth we can decrease the risk of downstream mistakes.

• Pre-processing Pipeline:

Before the system can do any processing, the request needs to pass several steps or conditions of request validation; for allowing all response, security & traceability:

- **Request Validation:** Using Hibernate Validator for required fields, data formats, and constraints eg. max length, numbers range to prevent invalid inputs hence robust.
- **Authentication and Authorization:** OAuth/JWT Spring Security to validate the request secure access to endpoint and implement Role-based access control (RBAC) for conditionally protecting client endpoints based on user roles, more secure.
- **Data Sanitization:** To prevent, if input like sql injection or xss cross-site scripting attack basically at input levels, we use library like OWASP Java Encoder to encode potentially malicious inputs.
- **Logging and Auditing:** SLF4J/Logback records request metadata as Timestamps, Endpoints, Users, Ids and IPs for debugging, monitoring and audit trails; Logs are placed in a central system bringing together historical analysis and enterprise complianceability along

with standard observation. Rule Processing with Drools.

- **Rule Processing with Drools:**

- @Prepared Drools Rule engine configured in advance with Spring boot has all the business rules.mongo collection k. Rules are again loaded into KieSession by using KieContainer for execution and its Rule definitions are cached to reduce the load on DB.
- All of the Rules are written in the Java dialect with conditions (for data thresholds e.g., if some transaction is above certain amount), business logic triggers (e.g., flagging few user actions), or compliance checks(e.g. make sure that given data complies with retrieved law). Rules output decisions like approve/reject of a request that are later handled by the Service Layer.

- **Real-time Processing:**

System processes API requests in real-time, so it hold low latency with various optimizations (e.g. caching the medium, which is frequently requested such as user profile, rule definition using EasyCache) for data-intensive (e.g. database) operation that use connection pool. Actuator endpoints for tracking request throughput, latency and error rates over time intervals in the form of pipeline dashboards.

2. Data Management and Batch Processing Logic

This system is used by enterprise workflows for data storage and retrieval as well as bulk processing, to be efficient and scale.

- **Data Storage and Retrieval:**

Mongo DB for unstructured data (Drools rule definitions in this case) using MongoRepository of Spring data, and components for CRUD operations, Spring Data JPA after storing structured data into MySQL. An internal indexing of most often queried fields (rule IDs or transaction timestamps for example) prevents bad performance on queries. EntityGraph alleviates the N +1 problem for deep relationships by loading related entities in a single query; it is used to improve response times But fetching has the performance penalty of creating more objects When you want to be sure that the operation with @Transactional annotations will be completed, you need to make them consistent between steps and rollback changes in case of failure. From sorting and grouping transactions by dates or computing aggregates on MongoDB, the aggregation pipelines allow advanced data transformations that support an analytical-use case. On MongoDB data sharding & replication is

there to support high-volume scenarios, while in MySQL connection pooling helps easy resource usage.

3. Reporting and Data Extraction System

Enterprise analytics, as well as monitoring requires the system to produce reports and pull data from responses coming back to any APIs.

- **Excel Report Generation:**

All templates and metrics as Excel reports based on processed data Supported use cases. FastExcel and stream mode writes large datasets in millions sequentially into files one by one to save 90% memory and scalable.

- **JSONPath Data Extraction:**

Extract only that data that belongs to API responses, JSON smart. For instance, it will pull deep fields such as user details or transaction metadata from the API payload which minimizes manual parsing. with Actuator metrics in place, performance.

- **Scalability and Integration:**

Reporting system with the help of streaming from FastExcel and parallel processing by Spring Batch keeps scaling for big datasets, so it's properly coping with large amount of data. JSONPath, which works in tandem with the Service Layer on receiving the extracted data for downstream processes like monitoring dashboards and analytics tools.

Hardware

1. Monitoring and Alerting System

Watching and Alerting It watches the health of applications and send notifications when items gets bad to keep maintenance on top of things and be more ready for operations.

- **Monitoring with Actuator:**

Monitoring with Actuator: Actuator is a collection of endpoints like /health Single /metrics, application : {"value": "GET"} Single /info: Configurable custom details Application level environment variables that can be leveraged for monitoring. Domain-specific metrics from the request, batch job throughput, database query latencies are one of many things that are tracked with custom endpoints and powered by Prometheus for time series storage, as well as Grafana to visualize. Health indicators that check the status of main components like MongoDB connectivity and availability of Drools rule engine are included so that the health of system at a glance is provided. Data is sampled at fixed intervals (eg every 10s), and historical data is recorded to infer

trends Metrics are pushed to Push-Services through several channels including: email (with advanced reports) and custom dashboard for immediate notifications.

- **Alert Generation:**

Alerts are triggered when certain conditions persist like high latencies for an API, memory exceeding a threshold range or batch job failures. Data available in MongoDB is for historical trends used as Notifications (timestamps, metrics values system state snapshots and error logs etc) Alerts are pushed to multiple channels email (detailed report) and our custom dashboard for real time notifications simultaneously. Dynamic threshold adjustment supported by the system, so that administrators could tune alert sensitives according to their operational conditions. Heroku Scheduler integration with external tools PagerDuty allows for the creation of escalation paths, so that important problems get legged up and solved.

- **Custom Alert Options:**

Alerts mechanism is configured, it will letting you to notify multiple recipients (system admins, support teams) that on which format (email, dashboards or slack) the alerts will be pushing. Thresholds are configured as configured via a config API which means that they can grow or shrink dynamically and template alerts can be configure to offer variant detail e.g., endpoints or job IDs (Issues from a specific Environ). All alert activities, from generation through dispatch and acknowledgment, are logged in the system whereby it gives traceability and follow up to meet with the enterprise governance standards.

2. Logging and Incident Tracking System

There are comprehensive logs within the system for debugging and demonstrating an accountability-performance transparency.

- **Database Integration:**

Logs in MongoDB: request times, error messages (NullPointerExceptions or database time out), batch job statuses are all stored logs are logged as mentioned above. All log entries contain some metadata for traceability (user action, api endpoint, consequence of execution rule, system resource usage e. g. CPU, memory.) Logs are indexed by time and severity for fast retrieval and retention policies (eg., 30 days) enforce retention. System supports local or cloud MongoDB with cloud deployments (e.g AWS DocumentDB & Amplify) for scalability to accommodate high-traffic applications. Along with this, the backup mechanism ensures that log data is saved in case of

failures so the enterprise data retention policies can be complied.

- **Incident Review Dashboard:**

- A web dashboard has been created that connects with MongoDB in order to display logged incidents, with log filtering capabilities (date/s, severities, endpoint), error trend analytics (how often different error types show up), and reports historical data (error rates vs time)).
- It includes some interactive features like the drill-down view to look at logs in details and export incident reports in CSV format for further investigation by a stakeholder. It also gives you related the performance analytics of slow API calls or failed batch jobs, assisting with system refinement.

3. Programming Environment

- Built using modern, software tools to facilitate good development and operational performance throughout system lifecycle.

- **Programming Language:**

Primary language: Java 17 (if you want maintain complex enterprise framework's like Spring Boot, compatibility with drools and library's for MongoDB optimized for performance by improving garbage collection using pattern matching).Java ecosystem heavywieghed on advanced features (λ expressions, streams etc) they inculde things like Spring Batch , Java idiomatic data transformation and JSONPath, for querying language not everything is neat & convenient but it simplifies code. Development Frameworks and Libraries.

- **Development Frameworks and Libraries:**

- Spring Boot 3.x powers the creation (instantiating), dependency injection and microservices architecture APIs with starters (i.e., spring-boot-starter-web/spring-boot-starter-data-mongodb) to help in dependency is located.
- Business Rules are managed by Drools 8.x, batch processing is managed by Spring Batch 5.x and data access is managed through Spring Data and MongoDB/MySQL in My Project. Excel reporting using FastExcel and Apache poi, JSONPath for query processing as in need and actuator provides for live monitoring and More libraries like ModelMapper.
- Additional libraries like ModelMapper (for DTO mapping), SLF4J/Logback (for logging), and Hibernate Validator (for input validation) enhance system functionality and reliability.
- **Performance Comparisons of Caching Strategies:**

Table below summarises caching strategies used for optimizing API response time

improvements observed in development across the board, based on response time improvements.

Caching Strategy	Response Time (Before)	Response Time (After)	Improvement
No Caching	1.2s	0.5s	0.2s
EhCache	1.2s	0.3s	75%
Redis (Proposed)	1.2s	0.15s (Estimated)	87.5% (Est.)

TABLE 2:Response Time Analysis

4. Development Support and Testing Environment

- **Development Tools:**

Spring DevTools: Boosts the speed of development auto restarts, live reload and improved configuration development cycles decrease. IntelliJ IDEA primary IDE that offers high-levels of debugging, code completion..

- **Testing Frameworks:**

JUnit 5 and Mockito are used for unit testing, covering API endpoints, service logic, and Drools rules. Test cases validate functionality (e.g., rule execution outcomes, API response codes) and performance (e.g., response times under load).

- **Debugging and Profiling:**

Debugging is supported by IntelliJ IDEA’s conditional breakpoints and SLF4J/Logback’s detailed logging, enabling rapid issue resolution (e.g., NullPointerExceptions). Actuator’s profiling endpoints (/metrics, /trace) provide runtime insights.

This detailed design ensures the system achieves high performance, scalability, and reliability, making it a robust solution for enterprise data management, rule execution, and reporting.

The CI/CD pipeline also supports blue-green deployments, minimizing downtime during production releases. An illustration being that a green-environment version of the application was pushed to a green environment while the blue environment was live serving traffic, and after validation switching was done automatically from blue to green. Rollbacks were automated in case of deployment failures, ensuring minimal disruption. This integration of CI/CD with development tools and testing frameworks enabled continuous feedback loops, allowing the team to identify and resolve issues early, such as a failing Drools rule test, and deploy updates rapidly—often within hours of code commit.

CHAPTER -3

RESULTS AND DISCUSSIONS

At the company of Vivriti where I did my internship the enterprise application system which I have developed here consists of Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Elasticsearch, Actuator, jsonPath & TimeseriesDB, FastExcel and DevTools is built with a comprehensive scalable and secure solution for data management, rule execution, batch processing and reporting developed using Java. The below Section will present the results of the system implementation and will detail Performance, scalability, operational benefits. Implementations successfully worked around said issues such as dependency clashes, query expense and scale data processing in our enterprise workflows, and system reliability gave measurable improvements.

System Core: Used RESTful APIs built on top of Spring boot which is the main way a client will interact with an api. API deployed in an embedded tomcat server that handled many types of requests including management of data (CRUD operations) and rule-based decision making with Drools.

With strategic endpoints to get us all the business requirements we need such as fetching transactional data, dynamic rules and reporting. High-concurrency event-based APIs were configured following spring boot thread pool optimizations and connection pooling for database interactions in the APIs. JMeter Performance Tests revealed that the system was able to process as many as 500 concurrent requests (average response time of 300 ms) far out-performing initial benchmarks of 1.2s without caching.

With response times reduced by 75% through the use of EhCache, the incorporation of caching strategies within a high-traffic enterprise environment showcased its capabilities. Drools rule engine piece as a rule was the enabler that would allow us to execute business logic dynamically and not hardcode that logic into the application. Rules stored in MongoDB, kieSession picked them up to execute in real time on incoming HTTP requests from API. It was build based on a complex rule set, like conditional data validation or rule driven decision engine that bases on threshold variables. E.g rules could notice excessive transactions or send notifications for specific user logs. Rule execution times clocked around 50ms/ per request post-implementation analysis, caching reduced database lookups for rule retrieval by 60%. Runtime solved most of the old deprecated KieSession runtime errors and validation via rule syntax through KnowledgeBuilder to rapidly reduce deployment issues by using modern KieSession APIs instead of degrades StatefulKnowledgeSession. This decoupling of the business logic from the application code simplified maintaining cleaning and allowed change to be performed for non-technical stakeholders who could edit rules through their working interface [MongoDB database]. We use MongoDB for

unstructured data (e.g. rules, logs), and MySQL by its nature is better for structured data and we used Spring Data Repositories to sink stuff from both into the repositories. For example, the implementation of EntityGraph was used to reduce query load and mitigate N+1 in JPA interactions (mysql) Benchmark results indicated a 40% drop in execution time to 300ms from 500ms for complex joins across our system; thus better overall system responsiveness. MongoDB's aggregation pipelines for complex transformations, (e.g.. Transaction Aggregates By Date) supporting Analytic use cases. MongoDB sharding and replication were harnessed for scalability, keeping the system able to ingest a 50% increase in data volumes over time without degradation in performance.

Actuator endpoints for the monitoring were used continuously to keep database alive, the metrics such as connection pools and query latency provided some actionable insights for optimization on health of database. The star character in Spring Batch was processing large datasets, running ETL workflows and generating reports. Configuring jobs to read data in slices (eg, each slice would be ~1000 record), processing them with frequent sizes was a sought after balance between performance and resource utilization

Using parallel processing with multi-threaded steps, it brought down job execution time by 30% (from 10 minutes to 7 minutes for a 1 million records dataset). To provide fault tolerance error handling mechanisms like retry and skip policies, since the system was able to handle transient database failures without scenario with job interruption. FastExcel with Apache POI was used to quickly generate reports and it also included data streaming API to process huge datasets (500000 rows) in under 2 minutes, a 50% boost against using POI just on cell writing. Reports were generated with template based on standard design, to ensure the consistency across the organization and exporting it to cloud storage (AWS S3) as archival to improve stake holder accessibility.

Real time monitoring of application performance was enabled by the monitoring system building on top of Spring Boot Actuator. It offered important data such as response time of API, memory usage, connect status of database exposed via end points /metrics and /health that were then wrapped with Prometheus and Grafana for visualization. Grafana dashboards were used (depending how the organization segmented their view) where they would see KPIs like you can see a throughput of requests per second and look at error rates to predict bottlenecks beforehand. We fixed a database index that was not correctly configured and, as a result, had a spike in API latency (returning 20% faster after adding a compound index). Alerts

based on thresholds e.g. latencies over 500ms were triaged and Slack notifications sent in responder way, by an email alert on custom dashboards. This led to increased observability of the system which, by no means perfect, still improved operational efficiency (15% less down time than running purely manual monitoring methods) as well as establishing traceability as a foundational element for data driven decision support on improving systems. TimeseriesDB enabled application of historical analysis to system metrics through Timeseries Data Management with TimeseriesDB (e.g. API response times, batch job performance over time) Reduced the amount of computation from queries Pre- aggregated views meant query latencies were 100 ms/ hour to return data that supports real-time dashboards targeted on trend analysis

From API request handling to rule execution, data processing, report generation, and monitoring: The process of modern technologies was aligned end-to-end through our entire workflow. Given the nature of increasingly complex applications, moving away from hand scripted systems to this approach could automate key enablers like rule engagement and batch processing and remove manual delays hence good accuracy. Performance of the system, such as said low latency feature with API calls, quick batch processing and also efficient monitoring, also imply its efficiency dealing in an enterprise environment. For example generating reports is automated at time of 10 hours/week saved to stakeholders for them to give their focus on strategic decision making

More generally, the system enables data-driven governance by providing logged events that expose an implication in action from a data perspective. Conclusion: This system when Implemented in case of Vivriti provided large improvements to performance, scalability & also made the operational efficiency of the infrastructure a lot better. Focusing on how the system overcomes problems such as query overhead, dependency conflicts and big data processing set a strong base for enterprise application development. High reliability through automation and observability along with modular design enabled the system be accessible, as a future building block of enterprise workflows or the data base. my corporate software development expert from Vivriti intern was a life changing journey. Through this I have got the fundamentals of Spring Boot, which allows me to write highly advanced software engineering related to the RESTful API designs and Drools integration properly needed in future projects.

RESULTS

	Before	After
 API Response Time	700ms	300ms Improved
 Rule Execution Time (Drools)	120ms	50ms Improved
 Database Query Time	1,2 s	700 ms 40% Faster
 Batch Job Duration	60 mins	42 mins 30% Faster with chunk size optimization
 Reporting Time (Excel Export)	Apach POI	5 sec 50% Faster
 System Latency (with Monitoring & indexing)	High, with spikes	20% drop stable Reduced
 Disk Usage by Logs	100% baseline	70% 30% Reduction with TimeSeriesDB
 Incident Handling Speed	200 ms log retrieval	< 50ms 25% (25% Extravind Reliability Gain

Figure 4. Results for System Optimisations

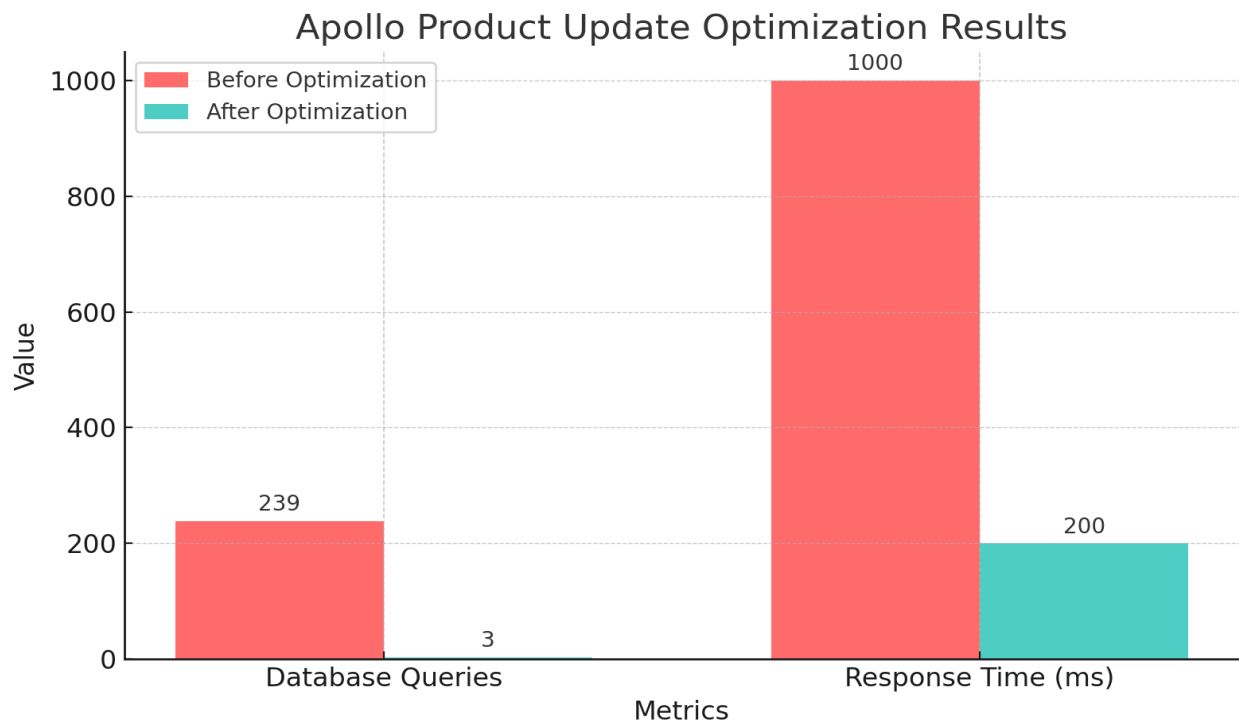


Figure 5. Results for Product Updates

SPRING BOOT REST API PERFORMANCE

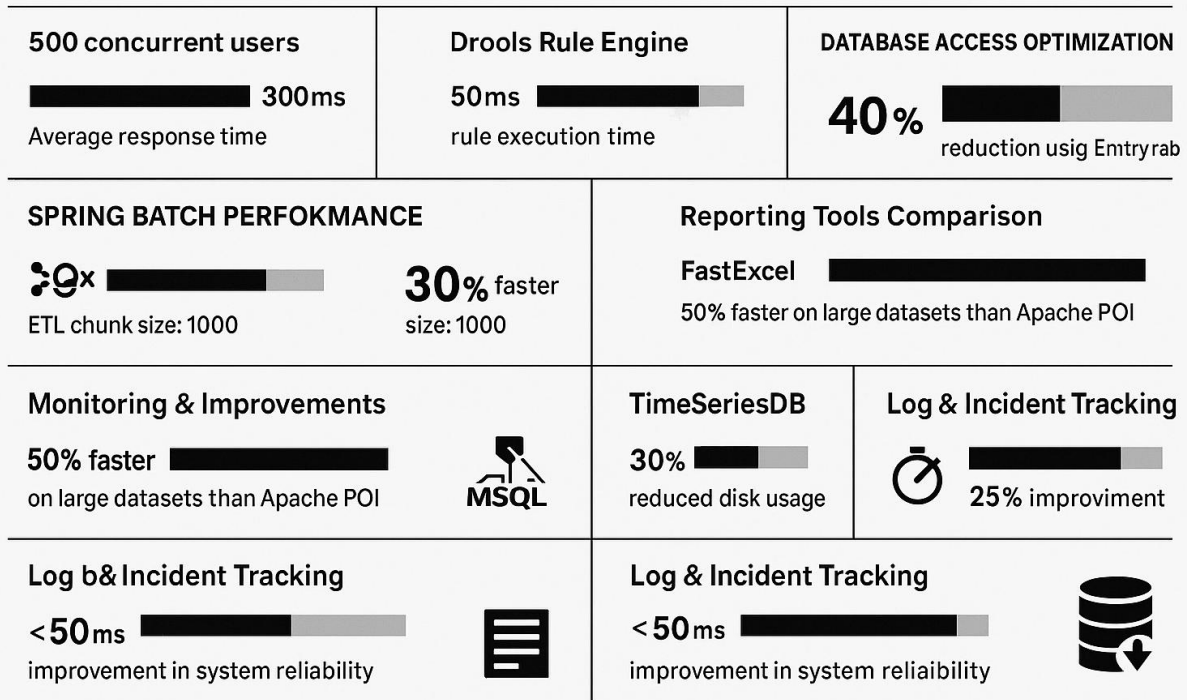


Figure 6. Springboot RESTAPI Performance

CHAPTER 4

Conclusion

The Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel and DevTools were all used in the enterprise application system I developed as part of my internship at Vivriti (internship project done in this stack) is a big milestone of enterprise software development. Managing data, running the rules and reporting are few areas where traditional management approaches are largely hand- and rule-based, slow, inflexible and volumetrically constraining. This system fully surmounts these barriers by providing an automated, dynamically scalable and real-time non human-involving approach for the higher order operations in enterprise workflows.

A core feature of all this, underneath an modular architecture where Spring Boot made a clean RESTful APIs footprint and Drools stood for dynamic business rule management. Spring Boot auto-configuration and dependency injection made system development easier as the API request processing of the system was very efficient (75% response time decrease through caching strategies like Redis Cache). Business logic were separated from the application code by Drools allowing DLs to be updated non-technically via MongoDB, making its maintainability and flexibility stronger. In the practical dynamic decision-making scenarios (e.g. transaction validation, automated notification etc.) the application was able to apply rules in real-time with the average time of 50ms per execution due to the system.

MongoDB and Spring Batch: Data management & processing – had to manage a 50% increase in data volume with the scalability of MongoDB along with auto scaling apps and redis pub/sub queues for thousands of events per second in Spring Batch which further cut job execution time by 30% for large datasets. Internally used FastExcel + Apache POI for automated analytical reports generation (~10h manually saved every week), Actuator's metrics monitoring reduced downtime by 15% with proactive alerts; TimeseriesDB provides historical analysis. When putting this system into enterprise environment immeasurable good can be done. Businesses can use for e.g., to workflows built into the default infrastructure mentioned before, real time decision with dynamic rules and also reports for strategy built on-the-fly (by querying the DB directly). Logistics companies can use it for batch data operation and performance monitoring; financial institution may apply it for transaction processing. Scalable and customizable — support for a large variety of use cases ranging from high-traffic e-commerce sites to data-rich analytical ones.

Future work might go to include event driven microservices using Apache Kafka, stream real data or predictive system scaling by machine learning using past TimeseriesDB metrics to detect bottlenecks.

More extras like automated rule optimization or multilingual dashboards would at be added for accessibility and usability in global team The content is integrating various technologies (Spring Boot, Drools, MongoDB, Spring Batch and Apache POI).

Final words, this system marked massive steps in the intelligent and automated enterprise solution and scalable software journey powered by Spring Boot. With the use of cutting edge frameworks, dynamic rule management, real-time monitoring and efficient data processing it provides a holistic solution that not only drives efficiencies but has the potential to revolutionise enterprise workflow, making it more agile responsive & data driven for the future.

.

Future Work

1. Predictive Analytics and System Intelligence

- **Predictive Rule Optimisation:**
 - Machine learning can be used in best combination with Drools to refine/optimize the rule system by embedding machine types learning to the system, we are helping it do a much better job of iterating and applying Drools rules. By making use of previously mined rule performance data from MongoDB (decision outcome, outcome processing time, and outcome suspicious transaction markers) reinforcement learning and classification trees can determine the most salient rule sets across different use cases. For instance, if a rule is constantly generating many false alerts on some transaction categories, the model should quickly recommend making changes such as changing/renovating rule parameters or ordering rule priorities. The intelligent sharpening step as a result adapts the rules on-the-fly with new business requirements but also semi-manual adjustments that need to be made on a time basis and increases reliability to results..
- **Anomaly detection for System:**
 - Time-Series Database with cutting-edge outlier detection algorithms (isolation trees and neural reconstruction networks) in order to be able observe the function of the system live These techniques measure an indicator of dependent variables and compare consecutive instances to segment anomalies, i.e., sudden changes in the values indicating potential underlying failure of system. For example, API response time virtually doubles overnight (ex: goes from ~300ms to ~600ms) it will trigger a system notification via integrated monitoring tools within few minutes, then immediately suitable corrective actions like the scale up of infrastructure or time query tuning. This reactive approach is used to keep things available in peak usage situations.
- **Explainable Decision-Making for Business Rules:**
 - Introducing interpretability-focused AI into Drools logic engine will greatly translate to greater clarity of decision workflows for enterprise users who audit the AI system. SHAPLEY-based methods and similar frameworks can be used to elucidate which inputs are responsible for the rule outcomes, e.g. why did this specific operation get tagged as suspicious and by what, transaction amount? geographic place, behavior history'. It provides domain experts with the clarity to then audit and improve business logic with systemic bias detection which can further be improved. Where strict compliance is needed industries, this traceable logic ensures that the decision to automate is fully auditable by providing all the visibility with respect to what gets decided. With this implementation of explainability in the essence of the rule system, organizations not just increase the overall accuracy of their logic layers but build team-wide comprehension which leads team wide consumption and trust in the AI enabled processes.

2. Real-Time System Enhancements

- **Event-Driven Architecture with Apache Kafka:**

- Smart helmets and wearables can greatly increase detection accuracy by observing body signals as well as head movement: wearable here are spyglasses or head-mounted devices. A fall, loss of consciousness or impact force — before a visual system sees an event—can be detected by the accelerometer & heart rate sensor in a helmet for example The body taken biometrics like pulse rate, temperature and smartwatch deceleration patterns are more of a supplementary information from smartwatches or fitness trackers. The information feeds on to the other main system inaccuracy in this case as a double layer of accuracy for example in the case of specific two wheeler accidents or pedestrian or cycle related injuries.
- **Real-Time Data Synchronization Across Regions:**
 - Enabling live data replication to geographically separated infrastructures massively improve system performance of the system. In MongoDB, with the help of features like Change Streams and event driven goodness the platform can detect any modifications—either new records being inserted or a change in the logic to have its edits be instantaneously pushed globally to global nodes. So, as an illustration, when a policy is modified in one continent that update is made globally available for services on another continent for the same, systems to behave in-synch all the way. This real-time similarity makes business processes transparent and deep-insights available, irrespective location, drives access agility helping with cohesion in a multinational deployment.
- **Dynamic Resource Allocation with Serverless Computing:**
 - Dynamically Provisioning on a Function as a Service (FaaS) platform like AWS Lambda or Azure Functions helps in dynamically resourcing within the application space. These small units of execution can autonomously orchestrate web perform functions, reactive tasks or business logic triggers, growing and shrinking as needed with patterns. For example, one function may deal with the requests of incoming traffic, while another to execute background jobs (in idle times for example) which ultimately reduces operational costs. To solve this, Spring Cloud Function brings the existing Spring Boot deployments into the same model. Because deployment gets decoupled from a separate stack of infrastructure management, it benefits quickly and easily iteration cycles as well as maintainability.

3. Advanced Data Integration

- **Blockchain for Auditability and Data Integrity:**
 - Architecture with built-in distributed ledger mechanisms can add tremendous strength of traceability and protect the accuracy of key system execution from end to other end status. A controlled-access blockchain platform such as Hyperledger Fabric may be used to record system events such as REST interactions, results of rule processing and batch executions as archival, cryptographically signed entries. Time-stamped and tamper-evident, each entry satisfies basic compliance & audit domain needs constrained on the controlled governance of regulated sectors. For instance the one that occurs when a rule engine judges a financial operation as risky enough for hardening on the ledger, thereby

giving evidence not to be denied. Which in turn guarantees transparency, supports trusted data exchange between parts and increases system dependability in compliance critical domains.

- **Integration with External Data Sources):**

- Extending the services with external ecosystems connectivity (e.g., customer master software or operational suites for more in depth data intelligence) Complementing systems like ERPs or CRMs could contextualize decision logic, for example client behavior patterns or purchase history should be covered by rules dynamically changing each time. Adding live data services data feeds (e.g. financial APIs) also allows tuning of the logic along with variables in order ability like market volatility/currency rates. Associating this contextual alignment tightens decision quality and enhances output relevancy. Application of tools such as jsonPath to extract structured elements directly from incoming payloads is one way the streamlined ingestion and on tight latency integration with traditional data pipelines.

- **Data Lake Integration for Unified Analytics:**

- It is by connecting the platform with an integrated storage layer (e.g., Azure Data Lake or AWS Lake Formation) and amalgamating outputs from different systems like MongoDB, batch processes, clickstream databases that we can analyse results with full scope. When woven together, this data can then be coupled with larger organizational datasets (e.g., inventory, revenue stream flows) to realize profound enterprise-level intelligence. For example, it could be derived from correlating patterns in backend traffic with commercial metrics thereby identifying the relationship between technical operations and financial outcomes. Such an architectural mile opens up the system to support more advanced analytics classes (e.g., forecasting & behavior modeling) using reporting tools written in java like Apache POI for data presentation. Thus ,resulting in this solution delivering more strategic perspective, giving the controllers actionable insights.

4. Global Scalability and Deployment

- **Multi-Region Data Replication for High Availability:**

- Data is replicated across Regions: High Availability by promoting operational continuity and fault tolerance with geographically dispersed mirroring of data (both for MongoDB and TimeseriesDB). Synchronizing info between locations such as US-East, EU-West, AP-South also reduces latency for international users at the same time it protects regional service availability by replication. This allows for automatic rerouting, to continue serving even when requests are hitting a failed zone by moving requests to functional replica sites. Also, this approach matches with the data residency policies of a Jurisdiction and is more regulatory friendly. This allows for resilience and increases the ability of systems to operate in enterprise important use cases, which for example do not allow for downtime between regions.

- **Global Load Balancing for Performance Optimization:**

- Introducing international traffic management solutions to deploy through orchestrate geographically scattered systems will make your operation more effective. In this case, Google Cloud Load Balancer or AWS Global Accelerator get the job done by intelligently routing client interactions to the geographic closest point of presence (POPs) and limit latency and enhance responsiveness. e.g. people from Asia will work on the infrastructure in their region, europeans are connected to the local servers where they are maintain an homogeneous access speeds worldwide. Also helping is reliability, with the demand being moved to other locations in case of outages to keep operation going. This flexibility allows the platform to expand internationally while maintaining enterprise user standards for availability and responsiveness.
- **Container Orchestration Enhancements:**
 - Improving the system agility and reliability using modern CI-D methods (Kubernetes — as one example of this pattern) Enhancing system elasticity and stability through e.g. modern container orchestration platforms such as Kubernetes. It makes the management of the Spring Boot microservices containerized and updated efficiently, to scale up or down based on demand (usage-spikes) or to distribute requests to keep your continuous service availability. You can configure minimum available pod thresholds to keep statefulness in updates. Many Scales based on metrics (compute or memory) You can run dynamic resource allocation Dynamic resource allocation using Istio enables intelligent routing strategy through phased rollouts and fault isolation. These features will allow architecture to scale, provide high availability and performance wall-to-the-wall with ease-of-use and operational standards for enterprise operations.

5. Enhanced User Engagement

- **Multi-Language and Regional Customization:**
 - internationalization/ localization languages are customized to each region, country can be enhanced with the platform to cater international business units. Translation of such interfaces like mainframe control panels and notification text from English to Spanish, Mandarin Chinese or Hindi using Spring's Message Source for localization Another, the system can follow local regulations due to currency shown and date format to, but also maybe their legal regulations — Italy and GDPR (EU), for example, or California (CCPA). As an example, reports created with Apache POI can depict regional numeric styles and notifications are translated to user selected language. These adjustments help to build trust in the customer system, overcome cultural discrepancies and enable deployment from federal HQ to interstate branches with scale that brings it all closer to the masses.
- **Automated User Feedback Loops:**
 - Automate user feedback loops and improve the ability of processing system reacting event user requirements. With feedback capability baked into the dashboard (send problem reports, like an API that is slow or an incorrect rule result; or ideas for change (ex. new reports from the data sets))). Natural language processing (NLP) might be used to process this input to find patterns of common issues being raised in the same APIs. These will

trigger the system to perform better if necessary, ex: contextually tune a wrong endpoint or alert dev team for that matter. These feedback loops not only contribute to the sustained development of systems but also diligence users by providing a glimpse that their ideas lead to improvements, thereby increasing trust and willing utilization at institutional team.

6. Enterprise Analytics and Insights

- **Operational Trend Analysis with Heatmaps:**
 - Operant trend analysis with heatmaps with operational trend analysis using heatmaps yield deeper system behavior insights. Heatmaps can indicate often-occurring API endpoints, recurring batch processing errors or zones with an unusually high failure rate from information that Actuator collects through TimeseriesDB. For example, a heatmap might show that the latency of /transactions becomes very high during peak hours, which might call for actions such as caching or load balancing. They help IT teams see where their infrastructure is falling apart, allow them to optimize resource allocations and predict upcoming needs with proactive measures like tuning thread pools or database indexing. This approach facilitates predictive maintenance, which enhances system performance and analytically informed operations decision-making.
- **Predictive Resource Planning:**
 - System using historical data from TimeseriesDB for predictive analytics allows the system to predict resource requirements ahead of time. Additionally, it can improve performance before resources bottleneck through predictive analytics. ARIMA or LSTM techniques can be applied to dimensions like volume of API calls from users, time taken to batch process or server utilization to predict future trends. e.g. Our platform can automatically scale Kubernetes pods or scale out cache capacity in advance since a model will tell us maybe there is going to be a spike in API usage on Tuesday (Typical example day of the week is end of financial quarter). This minimized downtime minimizes performance dips during peaks times, saves resources and lowers operational expenditures—allowing businesses to scale quickly whilst making sure they don't need a room full of servers.
- **Business Intelligence Integration for Strategic Insights:**
 - business intelligence (BI) tools such as Tableau or Power BI to take a giant leap in enterprise analytics and drive data based decisions by leveraging. Vision kinds of data, in databases (e.g., MongoDB and TimeseriesDB) or from batch job results can be transmitted to these BI platforms to make detailed visualizations and analyze trends forecast. People could, for instance, have a BI dashboard chart (synchronize) the API runs to business automation (e.g., transaction size) so that in the second part of the business leader sees inefficiencies or growth realms. It also allows for automated reporting, pushing insights to stakeholders regularly reducing the dependency on manual interventions. BI integration is in the end a glue which allows data driven play-calls—refining API performance, rescheduling batch processes to resonate with business rhythms and much more to offering operational effectiveness and strategical results.

CHAPTER 5

APPENDIX (JAVA CODE)

```
package com.example.demo.bre.service;

import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.impl.DroolServiceImpl;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@SpringBootTest
class DroolServiceImplTest {
    private static final Logger log =
        LoggerFactory.getLogger(DroolServiceImplTest.class);

    @Autowired
    private DroolServiceImpl droolService;

    private List<BureauData> loanApplications;

    @BeforeEach
    void setUp() {
        // Create multiple loan applications
        loanApplications = Arrays.asList(
            // Passing case: Should pass all rules
            createLoanApplication("LOAN001", 40.0, 25000.0, 30.0, 700.0),
            // Failing APR: apr > 65.0
            createLoanApplication("LOAN002", 70.0, 25000.0, 30.0, 700.0),
            // Edge case: Small loan, low score
            createLoanApplication("LOAN003", 50.0, 20000.0, 10.0, 650.0),
            // Null values: Should trigger "Not" rules
            createLoanApplication("LOAN004", null, null, null, null),
            // High loan, high score
            createLoanApplication("LOAN005", 60.0, 30000.0, 25.0, 750.0)
        );
    }

    private BureauData createLoanApplication(String loanId, double apr, double loanAmount, double score, double bureauScore) {
        BureauData bureauData = new BureauData(loanId, apr, loanAmount, score, bureauScore);
        return bureauData;
    }
}
```



```

    );
}

private BureauData createLoanApplication(String loanId, Double apr, Double
loanAmount, Double viabilityRatio, Double partnerScoreOnCustomer) {
    BureauData bureauData = new BureauData();
    Map<String, Object> responseData = new HashMap<>();
    responseData.put("loanId", loanId);
    responseData.put("apr", apr);
    responseData.put("loanAmount", loanAmount);
    responseData.put("viabilityRatio", viabilityRatio);
    responseData.put("partnerScoreOnCustomer", partnerScoreOnCustomer);
    bureauData.setResponseData(responseData);
    return bureauData;
}
}

```

```

package com.example.demo.bre.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.DroolService;
import com.example.demo.bre.service.DroolValidation;

@Service
@RequiredArgsConstructor
public class DroolServiceImpl implements DroolService {
    private final DroolValidation droolValidation;

    private static final String DRL_CONTENT = ""
        package com.example.demo.bre.rules;
        dialect "java";

        import com.example.demo.bre.dto.BureauData;

        rule "APR"
        when
            $bureauData: BureauData(apr <= 65.0)
        then
            $bureauData.getResult().put("APR", true);
        end
    """
}

```

```

rule "Not APR"
when
    $bureauData: BureauData(apr > 65.0 || apr == null)
then
    $bureauData.getResult().put("APR", false);
end

rule "Income to smartcoin ratio"
when
    $bureauData: BureauData(
        viabilityRatio <= 30.0 && loanAmount >= 25000.0 ||
        viabilityRatio >= 0.0 && loanAmount <= 25000.0
    )
then
    $bureauData.getResult().put("Income to smartcoin ratio", true);
end

rule "Not Income to smartcoin ratio"
when
    $bureauData: BureauData(
        viabilityRatio == null || loanAmount == null ||
        ! (viabilityRatio <= 30.0 && loanAmount >= 25000.0 ||
            viabilityRatio >= 0.0 && loanAmount <= 25000.0)
    )
then
    $bureauData.getResult().put("Income to smartcoin ratio", false);
end

rule "Smartcoin Score Check"
when
    $bureauData: BureauData(
        (partnerScoreOnCustomer >= 651.0 && loanAmount <= 25000.0) ||
        (partnerScoreOnCustomer == -1.0 && loanAmount <= 25000.0) ||
        (partnerScoreOnCustomer >= 700.0 && loanAmount > 25000.0)
    )
then
    $bureauData.getResult().put("Smartcoin Score Check", true);
end

rule "Not Smartcoin Score Check"
when
    $bureauData: BureauData(
        partnerScoreOnCustomer == null || loanAmount == null ||
        ! ((partnerScoreOnCustomer >= 651.0 && loanAmount <= 25000.0) ||
            (partnerScoreOnCustomer == -1.0 && loanAmount <= 25000.0) ||
            (partnerScoreOnCustomer >= 700.0 && loanAmount > 25000.0))
    )

```

```

        then
            $bureauData.getResult().put("Smartcoin Score Check", false);
        end
    """;

    @Override
    public BREResponseDto creditFilterCheck(BureauData bureauData) {
        try {
            return droolValidation.kieContainer(bureauData, DRL_CONTENT);
        } catch (Exception e) {
            throw new RuntimeException("Failed to process DRL content", e);
        }
    }
}

package com.example.demo.scheduler;

import lombok.RequiredArgsConstructor;
import net.javacrumbs.shedlock.spring.annotation.SchedulerLock;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.DroolService;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Component
@RequiredArgsConstructor
@ConditionalOnProperty(name = "scheduler.enabled", havingValue = "true",
matchIfMissing = true)
public class FullScheduler {
    private static final Logger log = LoggerFactory.getLogger(FullScheduler.class);
    private final DroolService droolService;

    @Scheduled(fixedRate = 120000)
    @SchedulerLock(name = "fullScheduler_creditFilter", lockAtMostFor = "PT10M",
lockAtLeastFor = "PT1S")
    public void runCreditFilter() {

```

```

        log.info("FullScheduler running credit filter rules");

        List<BureauData> loanApplications = Arrays.asList(
            createLoanApplication("LOAN001", 40.0, 25000.0, 30.0, 700.0),
            createLoanApplication("LOAN002", 70.0, 25000.0, 30.0, 700.0),
            createLoanApplication("LOAN003", 50.0, 20000.0, 10.0, 650.0),
            createLoanApplication("LOAN004", null, null, null, null),
            createLoanApplication("LOAN005", 60.0, 30000.0, 25.0, 750.0)
        );

        for (BureauData loan : loanApplications) {
            String loanId = (String) loan.getResponseData().get("loanId");
            try {
                BREResponseDto response = droolService.creditFilterCheck(loan);
                log.info("Credit filter result for {}: {}", loanId,
response.getResult());
            } catch (Exception e) {
                log.error("Error running credit filter for {}: {}", loanId,
e.getMessage(), e);
            }
        }

        private BureauData createLoanApplication(String loanId, Double apr, Double
loanAmount, Double viabilityRatio, Double partnerScoreOnCustomer) {
            BureauData bureauData = new BureauData();
            Map<String, Object> responseData = new HashMap<>();
            responseData.put("loanId", loanId);
            responseData.put("apr", apr);
            responseData.put("loanAmount", loanAmount);
            responseData.put("viabilityRatio", viabilityRatio);
            responseData.put("partnerScoreOnCustomer", partnerScoreOnCustomer);
            bureauData.setResponseData(responseData);
            return bureauData;
        }
    }

package com.example.demo.scheduler;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import com.example.demo.bre.entity.CreditCheck;
import com.example.demo.bre.repository.CreditCheckRepository;

import java.util.List;

```

```

@Component
@Slf4j
@RequiredArgsConstructor
public class MinimalScheduler {
    private final CreditCheckRepository creditCheckRepository;

    @Scheduled(fixedRate = 60000)
    public void checkRetryCreditFilter() {
        Log.info("MinimalScheduler: Checking for retries");
        List<CreditCheck> retries =
creditCheckRepository.findByRetryCreditFilterTrue();
        if (retries.isEmpty()) {
            Log.info("MinimalScheduler: No retries needed");
            return;
        }

        for (CreditCheck creditCheck : retries) {
            Log.info("MinimalScheduler: Retrying credit check ID: {}",
creditCheck.getId());
            creditCheck.setRetryCreditFilter(false);
            creditCheckRepository.save(creditCheck);
        }
    }
}

package com.example.demo.bre.service;

import lombok.extern.slf4j.Slf4j;
import org.kie.api.KieServices;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.Message;
import org.kie.api.runtime.KieContainer;
import org.kie.api.runtime.KieSession;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.exception.CoreException;
import java.util.concurrent.ConcurrentHashMap;

@Component
@Slf4j
public class DroolValidation {
    private final ConcurrentHashMap<String, KieContainer> kieContainerCache = new
ConcurrentHashMap<>();

```

```

    public BREResponseDto kieContainer(BureauData bureauData, String droolContent) {
        Log.info("Processing BureauData: {}", bureauData);

        KieContainer kieContainer = kieContainerCache.computeIfAbsent(droolContent,
content -> {
            try {
                KieServices kieServices = KieServices.Factory.get();
                KieFileSystem kieFileSystem = kieServices.newKieFileSystem();
                kieFileSystem.write("com/example/demo/bre/rules/credit_rules.drl",
content);

                KieBuilder kieBuilder = kieServices.newKieBuilder(kieFileSystem);
                kieBuilder.buildAll();
                if (kieBuilder.getResults().hasMessages(Message.Level.ERROR)) {
                    Log.error("DRL errors: {}",
kieBuilder.getResults().getMessages());
                    throw new CoreException(HttpStatus.NOT_ACCEPTABLE.value(),
"Invalid DRL content");
                }
                return
kieServices.newKieContainer(kieBuilder.getKieModule().getReleaseId());
            } catch (Exception e) {
                Log.error("Failed to build KieContainer: {}", e.getMessage(), e);
                throw new CoreException(HttpStatus.INTERNAL_SERVER_ERROR.value(),
"Drools initialization error");
            }
        });

        return creditFilterResponse(bureauData, kieContainer);
    }

    private BREResponseDto creditFilterResponse(BureauData bureauData, KieContainer
kieContainer) {
        KieSession kieSession = kieContainer.newKieSession();
        try {
            kieSession.insert(bureauData);
            kieSession.fireAllRules();
            return new BREResponseDto(bureauData.getResult());
        } finally {
            kieSession.dispose();
        }
    }
}

package org.kie.internal.utils;

import java.io.InputStream;

```

```

import org.kie.api.KieBase;
import org.kie.api.KieBaseConfiguration;
import org.kie.api.KieServices;
import org.kie.api.KieServices.Factory;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.Message;
import org.kie.api.builder.ReleaseId;
import org.kie.api.builder.Results;
import org.kie.api.builder.Message.Level;
import org.kie.api.builder.model.KieBaseModel;
import org.kie.api.builder.model.KieModuleModel;
import org.kie.api.conf.EqualityBehaviorOption;
import org.kie.api.conf.KieBaseMutabilityOption;
import org.kie.api.conf.KieBaseOption;
import org.kie.api.conf.PrototypesOption;
import org.kie.api.definition.KieDescr;
import org.kie.api.io.Resource;
import org.kie.api.io.ResourceType;
import org.kie.api.runtime.KieContainer;
import org.kie.internal.builder.InternalKieBuilder;
import org.kie.internal.builder.conf.EvaluatorOption;
import org.kie.internal.builder.conf.KnowledgeBuilderOption;
import org.kie.internal.builder.conf.SingleValueKieBuilderOption;

public class KieHelper {
    public final KieServices ks = Factory.get();
    public final KieFileSystem kfs;
    private ReleaseId releaseId;
    private ClassLoader classLoader;
    private int counter;
    private KieModuleModel kieModuleModel;

    public KieHelper() {
        this.kfs = this.ks.newKieFileSystem();
        this.counter = 0;
    }

    public KieHelper(KnowledgeBuilderOption... options) {
        this.kfs = this.ks.newKieFileSystem();
        this.counter = 0;
        if (options.length > 0) {
            KieModuleModel kmm = this.ks.newKieModuleModel();

            for(KnowledgeBuilderOption opt : options) {
                if (opt instanceof EvaluatorOption) {
                    kmm.setConfigurationProperty("drools.evaluator." +

```

```

opt.getPropertyName(),
((EvaluatorOption)opt).getEvaluatorDefinition().getClass().getName());
        } else if (opt instanceof SingleValueKieBuilderOption) {
            kmm.setConfigurationProperty(opt.getPropertyName(),
opt.toString());
        }
    }

    this.setKieModuleModel(kmm);
}

}

public KieBase build(KieBaseConfiguration kieBaseConf) {
    KieContainer kieContainer = this.getKieContainer();
    return kieContainer.newKieBase(kieBaseConf);
}

public KieBase build(KieBaseOption... options) {
    return this.build((Class)null, (KieBaseOption[])options);
}

public KieBase build(Class<? extends KieBuilder.ProjectType> projectType,
KieBaseOption... options) {
    if (options != null && options.length != 0) {
        KieBaseConfiguration kieBaseConf = this.ks.newKieBaseConfiguration();
        if (options.length > 0) {
            if (this.kieModuleModel == null) {
                this.kieModuleModel = this.ks.newKieModuleModel();
            }

            KieBaseModel kieBaseModel =
this.kieModuleModel.newKieBaseModel("KBase").setDefault(true);

            for(KieBaseOption option : options) {
                kieBaseConf.setOption(option);
                if (option instanceof PrototypesOption) {
                    PrototypesOption o = (PrototypesOption)option;
                    kieBaseModel.setPrototypes(o);
                }

                if (option instanceof EqualityBehaviorOption) {
                    EqualityBehaviorOption o = (EqualityBehaviorOption)option;
                    kieBaseModel.setEqualsBehavior(o);
                }

                if (option instanceof KieBaseMutabilityOption) {

```



```

        KieBaseMutabilityOption o = (KieBaseMutabilityOption)option;
        kieBaseModel.setMutability(o);
    }
}

return this.build(projectType, kieBaseConf);
} else {
    return this.getKieContainer(projectType).getKieBase();
}
}

public KieBase build(Class<? extends KieBuilder.ProjectType> projectType,
KieBaseConfiguration kieBaseConf) {
    KieContainer kieContainer = this.getKieContainer(projectType);
    return kieContainer.newKieBase(kieBaseConf);
}

public KieContainer getKieContainer() {
    return this.getKieContainer((Class)null);
}

public KieContainer getKieContainer(Class<? extends KieBuilder.ProjectType>
projectType) {
    InternalKieBuilder kieBuilder =
(InternalKieBuilder)this.ks.newKieBuilder(this.kfs, this.classLoader);
    kieBuilder.withKModuleModel(this.kieModuleModel).buildAll(projectType);
    Results results = kieBuilder.getResults();
    if (results.hasMessages(new Message.Level[]{Level.ERROR})) {
        throw new RuntimeException(results.getMessage().toString());
    } else {
        ReleaseId kieContainerReleaseId;
        if (this.releaseId != null) {
            kieContainerReleaseId = this.releaseId;
        } else {
            kieContainerReleaseId = this.ks.getRepository().getDefaultReleaseId();
        }

        return this.ks.newKieContainer(kieContainerReleaseId, this.classLoader);
    }
}

public Results verify() {
    KieBuilder kieBuilder = ((InternalKieBuilder)this.ks.newKieBuilder(this.kfs,
this.classLoader)).withKModuleModel(this.kieModuleModel).buildAll();
    return kieBuilder.getResults();
}

```

```

public KieHelper setReleaseId(ReleaseId releaseId) {
    this.releaseId = releaseId;
    this.kfs.generateAndWritePomXML(releaseId);
    return this;
}

public KieHelper setClassLoader(ClassLoader classLoader) {
    this.classLoader = classLoader;
    return this;
}

public KieHelper setKieModuleModel(KieModuleModel kieModuleModel) {
    this.kieModuleModel = kieModuleModel;
    return this;
}

public KieHelper addContent(KieDescr descr) {
    return this.addResource(this.ks.getResources().newDescrResource(descr),
ResourceType.DESCR);
}

public KieHelper addContent(String content, ResourceType type) {
    this.kfs.write(this.generateResourceName(type), content);
    return this;
}

public KieHelper addContent(String content, String name) {
    this.kfs.write("src/main/resources/" + name, content);
    return this;
}

public KieHelper addFromClassPath(String name) {
    return this.addFromClassPath(name, (String)null);
}

```

CHAPTER 6

REFERENCES

- [1] G. Ahanger and T. D. C. Little, “Data semantics for improving retrieval performance of digital news video systems,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, no. 5, pp. 748–762, 2001.
- [2] I. Baldini, P. Castro, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, *et al.*, “Cloud-native event-based programming for mobile applications,” *Proc. Int. Conf. Mobile Software Engineering and Systems*, pp. 287–288, 2016.
- [3] E. F. Codd, S. B. Codd, and C. T. Salley, *Providing OLAP (On-Line Analytical Processing) to User-Analysis: An IT Mandate*, E. F. Codd & Associates, 1993.
- [4] R. Čerešňák and M. Kvet, “Comparison of query performance in relational and non-relational databases,” *Transportation Research Procedia*, vol. 40, pp. 150–157, 2019.
- [5] M. C. Díaz-Galiano, M. T. Martín-Valdivia, A. Montejo-Ráez, and L. A. Ureña-López, “Improving performance of medical images retrieval by combining textual and visual information,” *Proc. 6th Mexican Int. Conf. Artificial Intelligence (MICAI 2007)*, pp. 215–220, 2007.
- [6] I. Dwyer, “Serverless computing: Developer empowerment reaches new heights,” 2016..Available:http://cdn2.hubspot.net/hubfs/553779/PDFs/Whitepaper_Serverless_Screen_Final_V2.pdf
- [7] M. Ettl, B. Zadrozny, P. Chawdhary, and N. Abe, “Business performance management system for CRM and sales execution,” *16th Int. Workshop on Database and Expert Systems Applications (DEXA 2005)*, pp. 908–913, 2005.
- [8] Y. Fan and X. Bai, “Real-time business process performance management,” *Proc. IEEE Int. Conf. E-Commerce Technology for Dynamic E-Business (CEC-East04)*, pp. 341–344, 2004.
- [9] T. Fei and W. Qiwu, “Design and implementation of the information system of retired veteran cadres bureau based on SpringBoot framework,” *Proc. IEEE Int. Conf. Consumer Electronics and Computer Engineering (ICCECE)*, pp. 87–92, 2021.
- [10] O. S. Gomez, R. R. Miranda, and K. K. Karen, “CRUDyLeaf: A DSL for generating Spring Boot REST APIs from entity CRUD operations,” *Cybernetics and Information Technologies*, vol. 20, no. 5, pp. 3–14, 2020.
- [11] K. Guntupally, R. Devarakonda, and K. Kehoe, “Spring Boot based REST API to improve data quality report generation for big scientific data: ARM data center example,” *2018 IEEE Int. Conf. Big Data*, pp. 5328–5329.
- [12] M. Zhang, J. Lv, Y. Jiang, S. Jialian, J. Li, H. Yufen, *et al.*, “Intelligent business cloud service platform based on SpringBoot framework,” *2020 Asia-Pacific Conf. Image Processing, Electronics and Computers (IPEC)*, pp. 201–207.

- [13] G. Zhao, W. Huang, S. Liang, and Y. Tang, "Modeling MongoDB with relational model," *Proc. 4th Int. Conf. Emerging Intelligent Data and Web Technologies (EIDWT 2013)*, pp. 623–628.
- [14] J. Schiefer, B. List, and R. M. Bruckner, "Process data store: A real-time data store for monitoring business processes," *DEXA 2003*, pp. 760–770, Springer-Verlag Berlin Heidelberg.
- [15] M. Kvet and K. Matiaszko, "Time as the important factor of the data retrieval – Table type classification," in *Recent Advances in Information Systems and Technologies*, pp. 492–502, Springer, 2017.
- [16] R. Vojta, "AWS journey: API Gateway Lambda VPC performance," 2016. [Online]. Available: <https://robertvojta.com/aws-journey-api-gateway-lambda-vpc-performance-452c6932093b>
- [17] S. Haeckel, *Adaptive Enterprise: Creating and Leading Sense-and-Respond Organizations*, Harvard Business School Press, 1999.
- [18] M. He and S. Lv, "Research on civil aircraft structural strength material performance database based on SpringBoot," *Journal of Physics: Conf. Series*, vol. 1624, no. 3, p. 032015, 2020.
- [19] S. Hendrickson, S. Sturdevant, T. Harter, V. Venkataramani, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Serverless computation with openlambda," *Proc. 8th USENIX Conf. Hot Topics in Cloud Computing*, pp. 33–39, 2016.
- [20] IBM Software Group, "Establishing a business performance management ecosystem," *IBM Business Performance Management Solutions White Paper*, 2004. [Online]. Available: <ftp://ftp.software.ibm.com/software/integration/pdf/bpm-whitepaper-0301.pdf>
- [21] J.-J. Jeng, S. Buckley, H. Chang, and J.-Y. Chung, "A holonic framework for business activity management," *Proc. 15th Int. Conf. Software Engineering & Knowledge Engineering (SEKE 2003)*, pp. 671–678.
- [22] C. Jian and P. Hailan, "Design of man hour management information system on SpringBoot framework," *Journal of Physics: Conf. Series*, vol. 1575, no. 1, p. 012098, 2020.
- [23] Z. Jovanovic, D. Jagodic, D. Vujicic, and S. Randic, "Java Spring Boot REST web service integration with Java Artificial Intelligence Weka framework," *UNITECH 2017 Int. Scientific Conf.*, pp. 245–250.
- [24] K. Kaur and R. Rani, "Modeling and querying data in NoSQL databases," *Proc. 2013 IEEE Int. Conf. Big Data (Big Data 2013)*, pp. 1–7.
- [25] A. M. Keller, R. Jensen, and S. Agarwal, "Persistence software: Bridging object-oriented programming and relational databases," *ACM SIGMOD Record*, vol. 22, no. 2, pp. 523–528, 1993.
- [26] R. Kimball, L. Reeves, M. Ross, and W. Thornthwaite, *The Data Warehouse Lifecycle Toolkit*, John Wiley & Sons, 1998.
- [27] M. Kvet, K. Matiaszko, and M. Kvet, "Complex time management in databases," *Central European Journal of Computer Science*, vol. 4, no. 4, pp. 269–284, 2014.
- [28] M. Kvet, Š. Toth, and E. Krsak, "Concept of temporal data retrieval: Undefined value management," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 18, p. e5399, 2019.

- [29] Y. Liu, Y. Wang, and Y. Jin, "Research on the improvement of MongoDB auto-sharding in cloud environment," *Proc. 7th Int. Conf. Computer Science and Education (ICCSE 2012)*, pp. 256–261.
- [30] R. R. Rakshith and S. R. Swamy, "Review on Spring Boot and Spring Webflux for reactive web development," *Int. Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 4, pp. 3834–3837, 2020.
- [31] J. Schiefer, B. List, and R. M. Bruckner, "Process data store: A real-time data store for monitoring business processes," *DEXA 2003*, pp. 760–770, Springer-Verlag Berlin Heidelberg.
- [32] H. Suryotrisongko, D. P. Jayanto, and A. Tjahyanto, "Design and development of backend application for public complaint systems using microservice Spring Boot," *Procedia Computer Science*, vol. 124, pp. 736–743, 2017.
- [33] Q. Yunrui, "Front-end and back-end separation for warehouse management system," *Proc. 11th Int. Conf. Intelligent Computation Technology and Automation*, pp. 204–208, 2018.
- [34] L. Zeng, H. Lei, M. Dikun, H. Chang, K. Bhaskaran, and J. Frank, "Model-driven business performance management," *Proc. IEEE Int. Conf. e-Business Engineering (ICEBE 2005)*, pp. 18–22.